

自考数据结构 串讲笔记

自考乐园版 课程代号：02331



课程说明



串讲目的

参考教材:

《数据结构》黄刘生主编

经济科学出版社

更多优质自考资料尽在百度贴吧自考乐园俱乐部

(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



课程说明

知识点：

线形表、栈、队列、数组、广义表、树、图、查找和排序。



第一章 绪论

§ 1. 基本概念与术语

数据结构:

是一门研究非数值计算的程序设计问题中计算机的操作对象及他们之间关系和操作等的学科。



（一）数据结构概念包括三个方面（三要素）

数据之间的逻辑关系（逻辑结构）

数据在计算机中的存储方式（存储结构）

实现数据操作的算法（数据的运算）



(二)、相关术语

1、数据：

能输入计算机并能计算机程序处理的符号的总称。

2、数据元素：

数据的基本单位。可以进一步细分为若干数据项，数据项是最小单位，不能再细分。

3、数据对象：

具有相同性质的数据元素的集合，是数据的一个子集。



4、

(1) 数据的逻辑结构：数据之间的关系。

A. 集合（无顺序）：

B. 线性结构（一对一）：

C. 树形结构（一对多）：

D. 网状、图结构（多对多）：



（ 2 ）数据的存储结构（物理结构）

数据结构在计算机中的表示。（ 2 种）

A. 顺序表示

借助数据在连续存储空间中的相对位置表示元素关系。

B. 链式表示

借助数据元素的存储地址的指针表示元素关系。



§ 2. 算法和算法分析

一、算法

定义：是对特定问题求解步骤的一种描述，是指令的有限序列。

特点：

有穷性

确定性

可行性

零个或多个输入

一个或多个输出



大O表示法

大O表示同级别

例： $f(n)=n^2$, $f(n)=1/2(n(n-1))$, $f(n)=(n-1)(n+2)$

均表示为： $O(n^2)$

加法规则： 若 $T1(n)=O(f1(n))$, $T2(n)=O(f2(n))$

则两段程序连在一起的时间代价为：

$$T(n) = T1(n) + T2(n)$$

$$= O(\max(f1(n), f2(n)))$$



例:

语句段	频度 $f(n)$	时间复杂度 $T(n)$
$x=x+1$	1	$O(1)$
for($j=1;j\leq 3n+5;j++$) $x=x+1$;	$3n+5$	$O(n)$
for($i=1;i\leq 3n;i++$) for($j=1;j\leq n;j++$) $x=x+1$;	$3n^2$	$O(n^2)$
$i=0$; while($x!=a[i] \ \&\& \ i\leq n$) $i=i+1$;	$n+1$	$O(n)$



求时间复杂度的原则

当重复执行次数与输入有关时, 计算平均值。

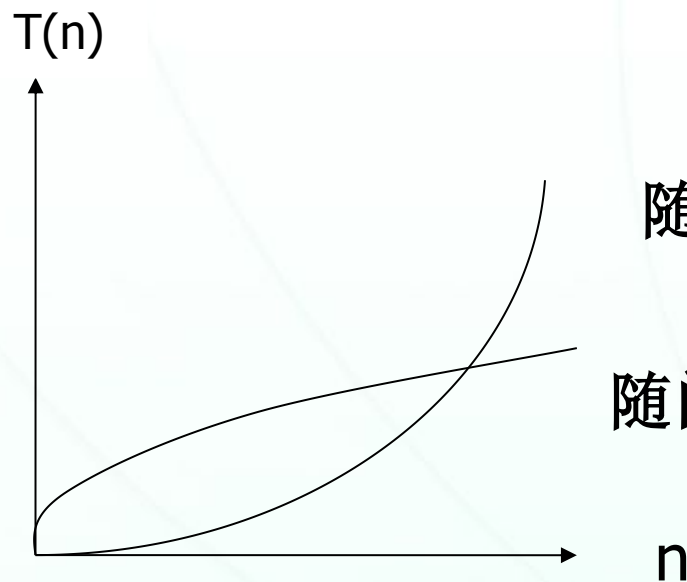
平均复杂度不易求时, 讨论最坏情况下的 $T(n)$ 。

更多优质自考资料尽在百度贴吧自考乐园俱乐部
(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



算法时间开销随问题规模变化趋势



随 n 增大, $T(n)$ 增加快, 算法坏

随问题规模 n 增大, $T(n)$ 趋缓, 算法好

$T(n)$ 的增大趋势:

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < \dots \\ < \dots < O(n^k) < \dots < O(2^n) < O(e^n) < \dots$$

算法的存储空间需求

算法空间开销与输入形式无关时，仅考虑辅加空间开销。

空间量依赖特定输入，讨论最坏情况下的空间开销。

（本章结束）



第二章 线性表

§ 1. 线性表的逻辑结构

一、线性结构

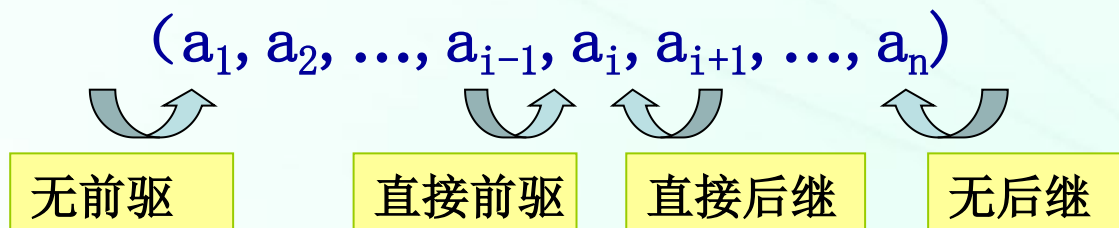
线性表是 n 个数据元素的有限序列

当 $n=0$ 时，称线性表为空表

当 $n>0$ 时，

线性表中各元素都有确定序号

线性表中各元素除第一个元素没有前驱、最后一个元素没有后继外，均具有唯一的前驱、后继元素



§ 2. 线性表的顺序存储结构

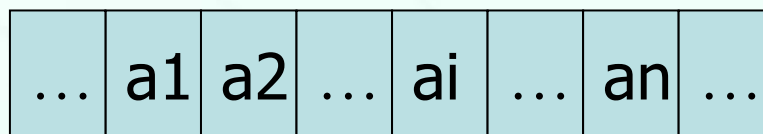
一、线性表的顺序存储

在计算机内开辟一片连续空间（存储单元）依次存放表中所有元素。

设线性表为A (a1, a2, ..., ai, ..., an)，表中的一个元素占用L个存储单元，第一个元素a1的起始地址是Loc(a1)，则第i个元素的起始地址为：

$$\text{Loc}(a_i) = \text{Loc}(a_1) + (i-1) * L$$

连续空间



§ 3. 线性表的链式存储结构

一、单链表

定义：

存储空间上一个结点对应线性表上一个元素，结点分为两个字段（或两个域）。一个字段存放元素的数据值；另一个字段存放指针，指向后继元素。

结点：

Data	Pointer
------	---------



两个概念：

头指针：

指示链表中第一个结点。

头结点：

在第一个结点之前附设的结点，其指针域指向链表中第一个结点。



在链表中第 i 个结点前插入新结点 b （前插）

算法分析:

基本操作：查找第 $i-1$ 个元素

当 $1 \leq i \leq n$ 频度为： $i-1$

当 $i > n$ （最坏，即 i 不合法） 频度为： n

$T(n) = O(n)$



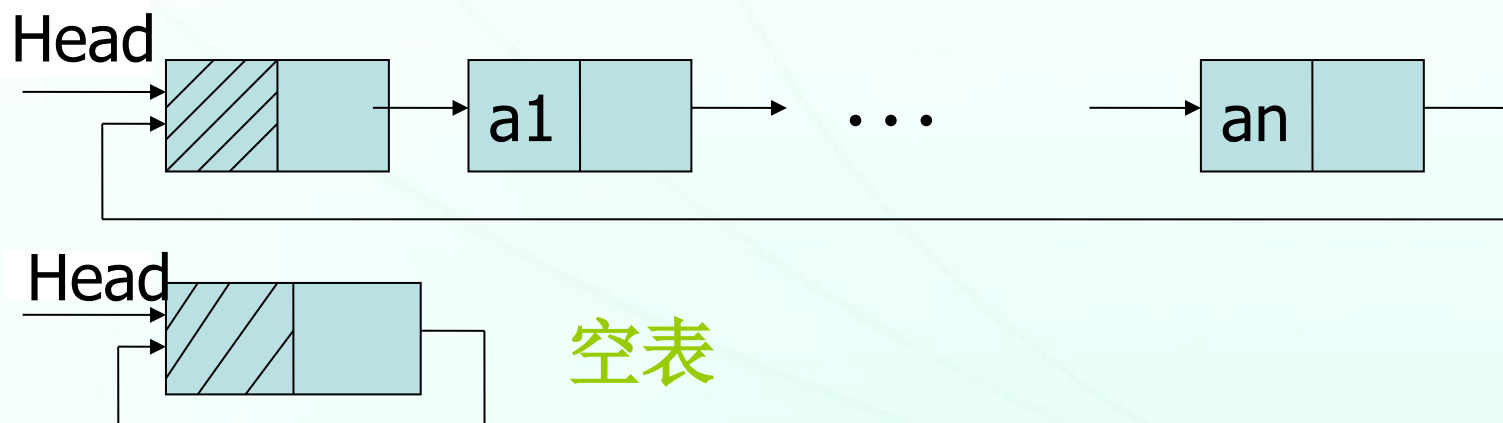
二、循环链表

特点

尾结点的next指针指向头结点，可以设头结点和尾结点指针。

优点

可迅速找到头、尾结点。



三、双向链表和双向循环链表

特点：

在结点中增加一个指向前趋的指针域。

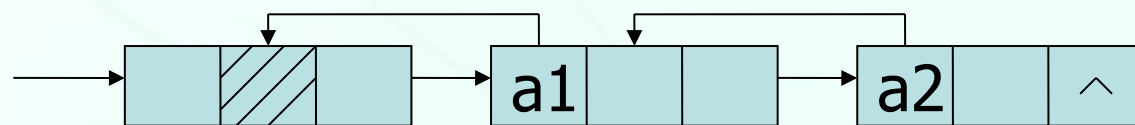
优点：

可迅速找到结点前趋。

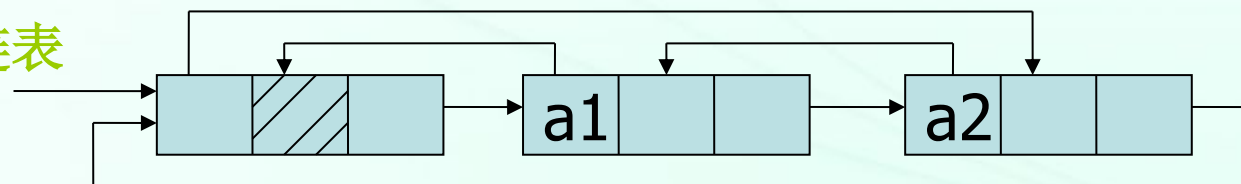
缺点：

增加存储空间。（每个结点增加了一个指针域）

双向链表



双向循环链表

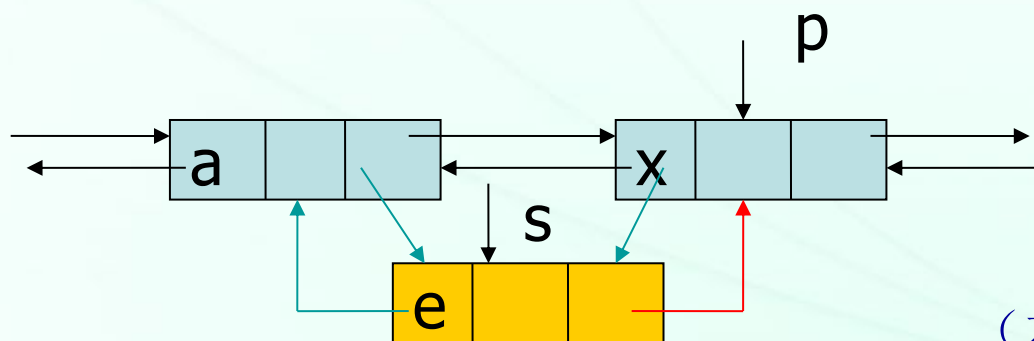


2、基本操作

(1) 插入：在链表的x结点前插入元素e

步骤：

- 在链表中查找x结点
- 申请结点空间s， $s \rightarrow \text{data} = e$
- 先做 $s \rightarrow \text{prior} = p \rightarrow \text{prior}$; $p \rightarrow \text{prior} \rightarrow \text{next} = s$;
- 后做 $s \rightarrow \text{next} = p$; $p \rightarrow \text{prior} = s$;



(本章结束)

(2) 删除：删除双向循环链表中的结点x

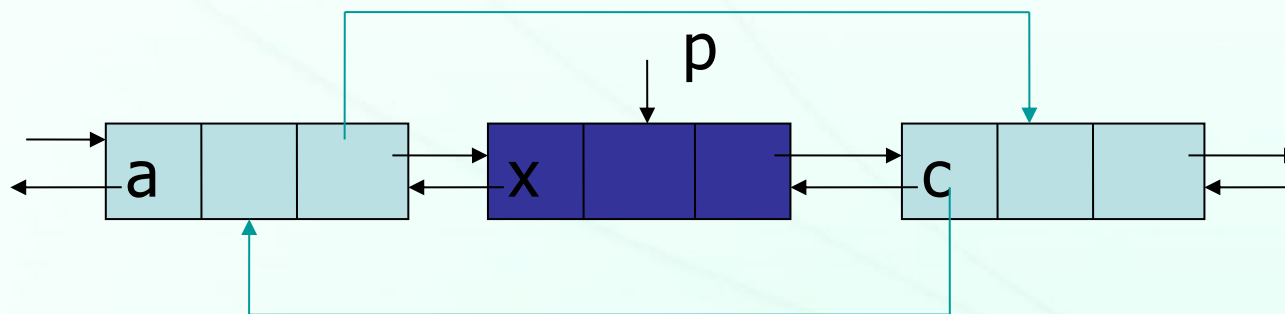
步骤：

a. 在链表中查找x结点

b. 删除： $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next};$

$p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior};$

c. $\text{free}(p)$



第3章 栈与队列

§ 1. 栈

一、栈的描述

1. 栈的定义

2. 栈的特点：后进先出（LIFO）

1, 2, 3, 4顺序进栈，有多少种出栈的情况？

$$S_n = s_0 s_{n-1} + s_1 s_{n-2} + \dots + s_{n-2} s_1 + s_{n-1} s_0$$

$$S_0 = 1, s_1 = 1$$

更多优质自考资料尽在百度贴吧自考乐园俱乐部

(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



例：

一个栈的输入序列为1， 2， 3， 4， 下面哪一个序列不可能是这个栈的输出序列？（ ）

A. 1， 3， 2， 4

B. 2， 3， 4， 1

C. 4， 3， 1， 2

D. 3， 4， 2， 1



二、栈的表示和实现

1、栈的顺序存储结构——顺序栈

顺序栈:

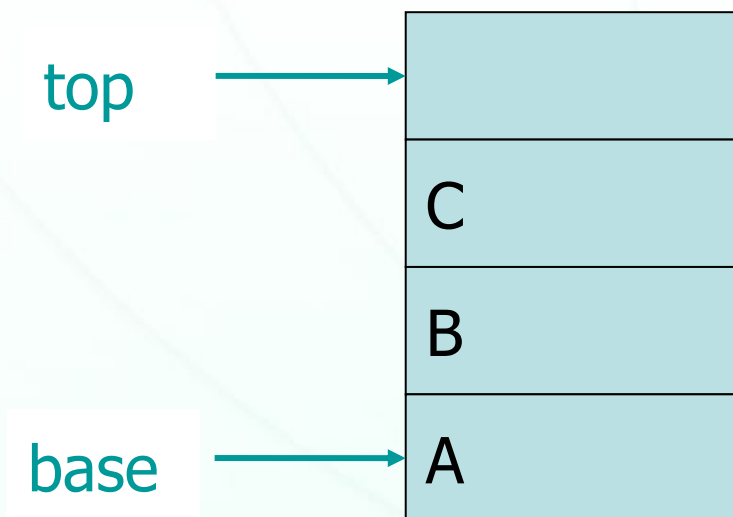
开辟一组地址连续的存储单元，依次存放自栈底到栈顶的数据元素。

设top指针指示栈顶元素的当前位置。



基本操作

栈示意图:



新元素入栈顶时：先入栈，

再移指针 $top = top + 1$

删除元素时：

先移指针 $top = top - 1$ ，

后删栈顶元素

栈的长度： $top - base$

2、链栈

定义：

采用链式存储结构的栈，并由其栈顶指针惟一确定。

设 ls 为栈顶指针，栈 $= (a_1, a_2, \dots, a_n)$ ， a_1 为栈底元素， a_n 为栈顶元素。



§ 2. 队列

一、队列描述

1、队列定义

2、队列特点：先进先出（FIFO）



二、队列的顺序存储表示

1、顺序队列

用一组地址连续的存储单元依次存放从队头到队尾的元素。

设队头指针为front，队尾指针为rear。

约定：当 $\text{front}=\text{rear}=0$ ，表示空队列；

front指向队头元素；

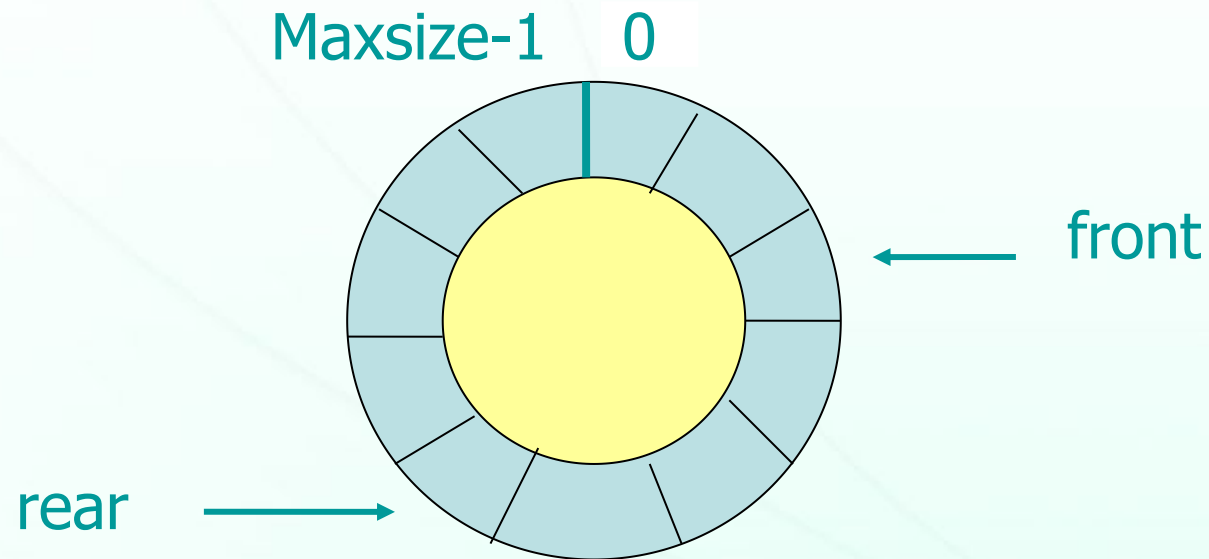
rear指向队尾元素的下一个位置。



2、循环队列

假想：

将队列的头、尾相连形成一个环，构成循环队列。并引入数学中的模运算计算队头和队尾指针。



基本操作:

入队操作:

```
Q. base[Q. rear]=x;
```

```
Q. rear=(Q. rear+1)%MAXQSIZE;
```

出队操作:

```
x=Q. base[Q. front];
```

```
Q. front=(Q. front+1)%MAXQSIZE;
```



问题:

循环队列的队空、队满条件?

举例说明:

矛盾:



矛盾

对空: $\text{front} = \text{rear}$

队满: $\text{front} = \text{rear}$

解决方法:

少用一个存储空间

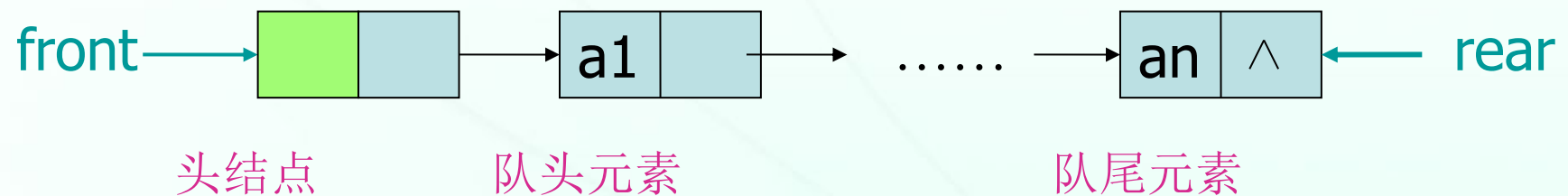


三、链队列

定义：

用链表示的队列简称为链队列。

设两个指针front、rear分别指示队头和队尾。为了链队列的操作方便，增加一个头结点，front指向头结点，rear指向队尾元素。如图所示：



§ 3. 递归算法设计

一、递归算法设计

递归过程：

一个直接调用自己或通过一系列的过程调用语句间接地调用自己的过程称为递归过程。

直接递归调用

间接递归调用



1、递归算法的设计思想

分治思想：

对一个较大问题可分解成属性相同、规模较小的问题，在小问题求解之后，通过组合，求得原来较大问题的解。

递归定义：

基本项：描述递归过程的终结状态，即可直接求解状态。

归纳项：描述如何实现从当前状态到终结状态的转化，即子问题与原问题之间的转化。



例1：求 $n!$

用分治思想分析, 得到递归定义:

基本项：当 $n=1$ 时, $f=1$ 。（直接求解状态）

归纳项：如果能求得 $(n-1)!$ ，原问题 $n!$ 则迎刃而解。 $(n-1)!$ 与 $n!$ 问题的属性特征相同，只是规模小了。所以，归纳项是设法求 $(n-1)!$ 。



2、递归算法设计

在程序设计中，什么情况下使用递归算法进行程序设计呢？考虑以下三个方面的因素：

当问题的定义是递归的

解决问题的数据结构是递归的

解决问题的过程是递归的

更多优质自考资料尽在百度贴吧自考乐园俱乐部
(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....

(本章结束)



第四章 串

§ 1 . 基本概念与基本运算

一、串

1、串定义：有零个或 n 个字符构成的有限序列。

记为 $S = 'a_1a_2 \cdots a_n'$ ($n \geq 0$) ; 其中 S 是串名, $a_1a_2 \cdots a_n$ 是串值, a_i ($i \leq n$ 且 $i \geq 1$) 可以是字母、字符、数字, n 是串长度, 当 $n=0$ 时为
空串。



§ 2. 串的存储结构

一、顺序存储结构

（或称静态存储结构：编译时进行空间分配）

定义：用一组**地址连续**的存储单元存储串的字符序列。

两种编址形式：

字节编址：一个**字节**存一个字符。（如早期的pc机）

字编址：一个**字**存一个字符。

非紧缩格式：每个字存放一个字符。

紧缩格式：每个字节存放一个字符。

C语言仅有紧缩格式：`char str[max];`



概念：存储密度

$$\text{存储密度} = \frac{\text{串值所占的存储位}}{\text{实际分配的存储位}}$$



顺序存储结构的特点

预先分配串最大长度，有可能造成存储密度低。

使串的某些操作受到很大限制。如置换、联接等操作。



二、动态存储结构

（在运行时进行存储空间的分配）

定义：

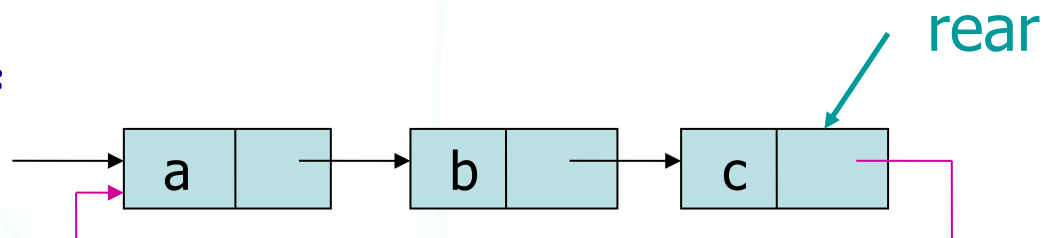
用一组任意的存储单元（可以连续，可以不连续）存放串的字符序列。



1、链式存储结构

每个结点存放一个字符。

图示：

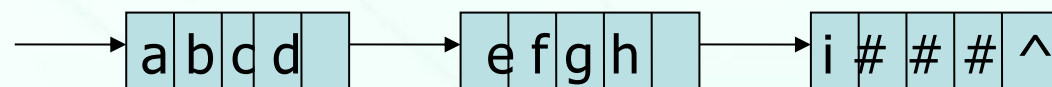


存储密度= $1/(1+4)=20\%$ （设指针占用4字节）

特点：密度低，存储空间利用率低，操作简单。

每个结点存放多个字符——块链结构。

图示：



存储密度= $4/(4+4)=50\%$ （设指针占用4字节）

特点：存储密度大，节省空间，操作复杂。

链式结构的特点：结点的选择非常重要。

§ 3. 串的模式匹配算法

模式匹配:

子串的定位操作通常称为串的模式匹配，是各种串处理系统中重要的操作之一。

$\text{Index}(S, T, \text{pos})$

其中S为主串，T被称为**模式串**。

若T为S子串，则返回T在S中第pos个字符后首次出现的位置。

若T不为S子串，则返回0。



模式匹配

模式匹配：子串在主串中的定位操作 ($m \leq n$)。

$t = t_0 \ t_1 \ t_2 \ \dots \ t_{n-1}$

目标

$p = p_0 \ p_1 \ p_2 \ \dots \ p_{m-1}$

模式

从目标 t 中查找与模式 p 完全相同子串的过程。

朴素的模式匹配。



1、朴素的模式匹配

模式匹配的最简单的做法是：用p中的字符依次与t中的字符比较：如果 $t_0 = p_0$ ， $t_1 = p_1$ ，...， $t_{m-1} = p_{m-1}$ ，则匹配成功，调用求子串的操作 $\text{subStr}(t, 1, m)$ 即是找到的子串。否则必有某个 i ($0 \leq i \leq m-1$)，使得 $t_i \neq p_i$ ，这时可将p右移一个字符，用p中字符从头开始与t中字符依次比较；如此反复执行，直到下面两种情况之一：或者到达某步时， $t_i = p_0$ ， $t_{i+1} = p_1$ ，...， $t_{i+m-1} = p_{m-1}$ ，匹配成功， $\text{subStr}(t, i+1, m)$ 即是找到的（第一个）与模式p相同的子串；或者一直将p移到无法与t继续比较为止，则匹配失败。

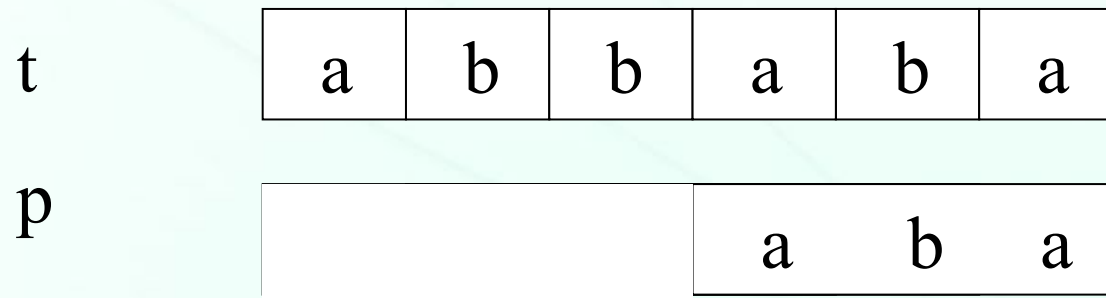


图3.3 朴素的模式匹配过程

说明:

S串下次比较位置: $i-j+2$

$j>T[0]$ 时说明匹配成功。

匹配成功的返回值: T中第一个字符在S中的位置。

(本章结束)



第五章 数组和广义表

§ 1. 数组定义

一、数组定义 (n 维数组)

当 $n=1$ 时，称为一维数组。

当 $n>2$ 时，称为多维数组。一个 n 维数组是由若干个 $n-1$ 维数组构成的。

更多优质自考资料尽在百度贴吧自考乐园俱乐部
(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



举例：二维数组 $m \times n$

$$A_{m \times n} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

可以看成由若干个一维数组组成，这些一维数组或按行或按列向量构成



按列向量:

$A_{m \times n} =$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

n个一维数组



按行向量:

$$A_{m \times n} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

$$A_{m \times n} = ((a_{11} \ a_{12} \dots a_{1n}), (a_{21} \ a_{22} \dots a_{2n}), \dots, (a_{m1} \ a_{m2} \dots a_{mn}))$$

m个一维数组



§ 2. 数组的顺序存储表示

问题:

存储单元是一维的结构，而数组是多维结构，如何用一组连续存储空间存放数组的数据元素呢？

解决方法:

存储映射策略;

行优先存储;

列优先存储。



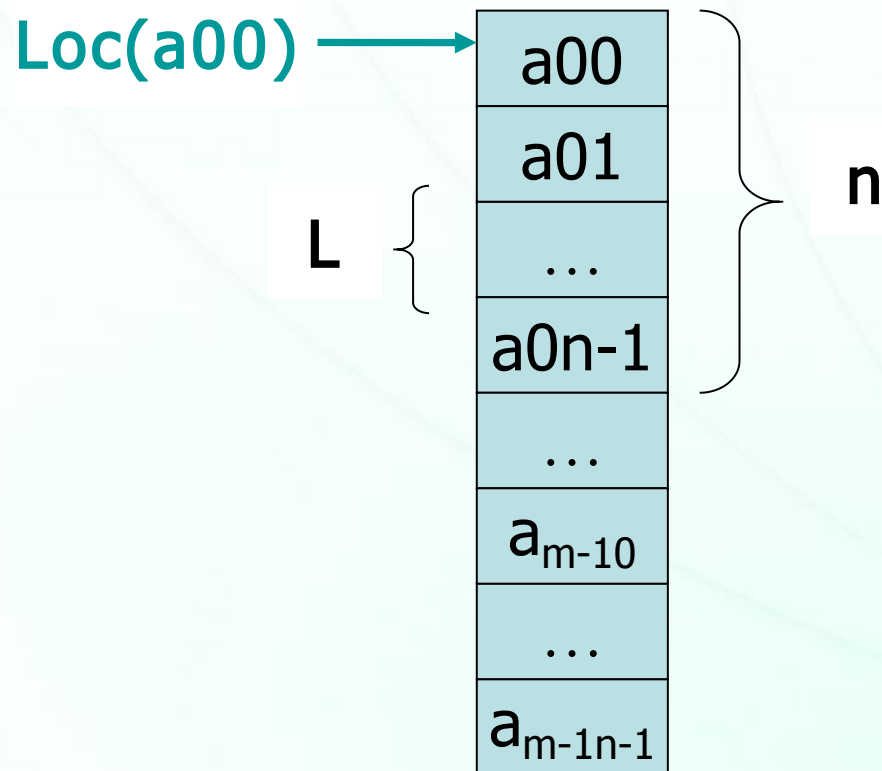
数组元素满足 $1 \leq i \leq m, 1 \leq j \leq n$

设每个数据元素占 L 个存储单元，第一个元素 a_{00} 存储地址是 $\text{Loc}(a_{00})$ ，问 a_{ij} 在空间上的存储地址是什么？



1、行优先存储

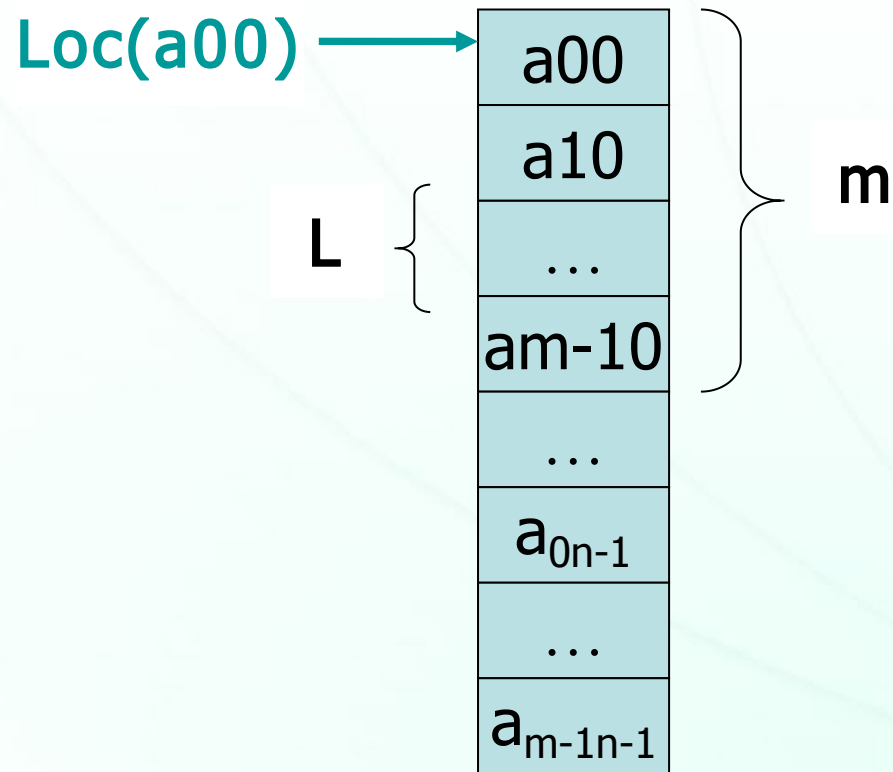
- $\text{Loc}(a_{ij}) = \text{Loc}(a_{00}) + (i * n + j) * L$



假设：行、列的下界为0

2、列优先存储

- $\text{Loc}(a_{ij}) = \text{Loc}(a_{00}) + (j * m + i) * L$



假设：行、列的下界为0

§ 3. 矩阵的存储及运算

一、一般矩阵的存储

通常用二维数组存储矩阵。

特点：

矩阵中的每个元素占用二维数组中的相应位置。



二、特殊矩阵的存储

特殊矩阵:

矩阵中相当一部分元素取值相同或都为0或元素在矩阵中分布有一定规律。

特殊矩阵的存储方式：通常采用压缩存储。

压缩存储:

为多个值相同的矩阵元只分配一个存储空间；对0元不分配空间。

更多优质自考资料尽在百度贴吧自考乐园俱乐部

(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



1、对称矩阵

定义:

对于一个 $n \times n$ 方阵, 且 $a_{ij}=a_{ji}$ ($1 \leq i \leq n, 1 \leq j \leq n$), 则称为对称矩阵。

特点:

矩阵中元素关于对角线元素对称。

存储方式:

采用三角存放法。即指存储对角线及对角线以下或对角线以上的元素。对于 n 阶对称矩阵, 仅用 $n(n+1)/2$ 个存储单元。



$$A_{n \times n} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix}$$

存储方式：按行优先

k

a_{11}	a_{21}	a_{22}	a_{31}	$\dots$	a_{ij}	$\dots$	a_{n1}	$\dots$	a_{nn}
----------	----------	----------	----------	---------	----------	---------	----------	---------	----------

$$K = \begin{cases} 1/2i(i-1) + (j-1) & i \geq j \\ 1/2j(j-1) + (i-1) & i < j \end{cases}$$

2、三角矩阵

特征:

n 阶矩阵，对角线下（或上）的元素均为零或各元素相等于某一个常数 C 。

存储方式:

采用三角存放法。即指存储对角线及对角线以下的元素。除此之外，在增加一个存储常数 C 的存储单元。对于 n 阶三角矩阵，仅用 $n(n+1)/2+1$ 个存储单元。



$$A_{n \times n} = \begin{pmatrix} a_{11} & & & \\ a_{21} & a_{22} & & C \\ \dots & \dots & \dots & \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix}$$

存储方式：按行优先

k

a ₁₁	a ₂₁	a ₂₂	a ₃₁	...	a _{ij}	...	a _{n1}	...	a _{nn}	C
-----------------	-----------------	-----------------	-----------------	-----	-----------------	-----	-----------------	-----	-----------------	---

$$K = \begin{cases} 1/2i(i-1) + (j-1) & i \geq j \\ 1/2n(n+1) & i < j \end{cases}$$

3、对角线阵（对角矩阵）

特征：

$n \times n$ 方阵，若他的全部非0元都集中在以主对角线为中心的带状区域内。

存储方式：

压缩存储。



三、稀疏矩阵的存储及运算

稀疏矩阵：

矩阵中的许多元素为0，并且0元素的个数远远大于非0元的个数。



1、三元组表

三元组表是稀疏矩阵的压缩存储方法之一，是顺序存储结构。

稀疏矩阵中的一个非0元用一个三元组 (i, j, value) 表示；其中：

i 表示该非0元在矩阵中的行号；

j 表示该非0元在矩阵中的列号；

Value 表示该非0元的值。

整个矩阵中每个非0元的三元组共同构成该稀疏矩阵的三元组表。



§ 3. 广义表

一、广义表

定义： 设广义表 $LS=(\alpha_1, \alpha_2, \dots, \alpha_n)$ ， n 是表长。

(1) 当 $n=0$ 时，称广义表 LS 是空表；

(2) 当 $n>0$ 时， $\alpha_1, \alpha_2, \dots, \alpha_n$ 是满足线性有序关系的，且 α_i 可以是某一数据对象元素，也可以是一个表结构。前者称为**原子**，后者称为**子表**。（在表示习惯上，广义表名称用大写字母表示，原子用小写字母表示）

特点：

广义表的结构是递归的。



举例:

广义表	表长
LS1=()	n=0(空表)
LS2=(a)	n=1(只有原子)
LS3=(a,(b))	n=2
LS4=(a,b)	n=2
LS5=(a,(b,(c)))	n=2
F=(a,F)	n=2(递归表)



广义表的术语:

当广义表非空时:

(1) 广义表的**表头**: 表中第一个元素;

(2) 广义表的**表尾**: 除了表头之外, 表中其余元素构成的子表。

特点:

表头可以是原子或子表, 而表尾一定是子表。



举例:

$LS = ((a, b), c)$

Head: (a, b) Tail: (c)

$LS = (a)$

Head: a Tail: () 空表

$LS = ((), (e), (a, (b, c, d)))$

Head: () Tail: ((e), (a, (b, c, d)))



广义表运算：设广义表 $LS = (\alpha_1, \alpha_2, \dots, \alpha_n)$

取广义表表头 $Head(LS) = \alpha_1$

取广义表表尾 $Tail(LS) = (\alpha_2, \dots, \alpha_n)$



二、求广义表深度

1、广义表深度定义：广义表括号重数

例： $LS = ()$ 深度：1

$LS = (())$ 深度：2

$LS = (a)$ 深度：1

$LS = (a, (b, c))$ 深度：2



例：求 $E=(A, B, C)$ 的深度

其中： $A=()$, $B=(e)$, $C=(a, (b, c, d))$

解： $\text{Depth}(E)=1+\max\{\text{Depth}(A), \text{Depth}(B), \text{Depth}(C)\}$

$$\text{Depth}(A)=1$$

$$\text{Depth}(B)=1+\max\{\text{Depth}(e)\}=1$$

$$\text{Depth}(C)=1+\max(\text{Depth}(a), \text{Depth}((b, c, d)))$$

$$\text{Depth}(a)=0$$

$$\begin{aligned}\text{Depth}((b, c, d)) &= 1+\max\{\text{Depth}(b), \text{Depth}(c), \text{Depth}(d)\} \\ &= 1\end{aligned}$$

$$\text{Depth}(C)=1+\max\{0, 1\}=2$$

$$\text{Depth}(E)=1+\max\{1, 1, 2\}=\mathbf{3}$$

(本章结束)



第六章 树

§ 1 . 树的定义和术语

一、树的定义

二、树的术语

结点度、树的度、结点层次、树的深度、家族树、有序树。

更多优质自考资料尽在百度贴吧自考乐园俱乐部

(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



§ 2. 二叉树

一、二叉树的相关概念

1、二叉树定义

树中每个结点至多有两棵子树，且子树有序。

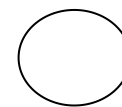
（根据二叉树的定义，二叉树的结构是递归的）

2、二叉树的基本形态

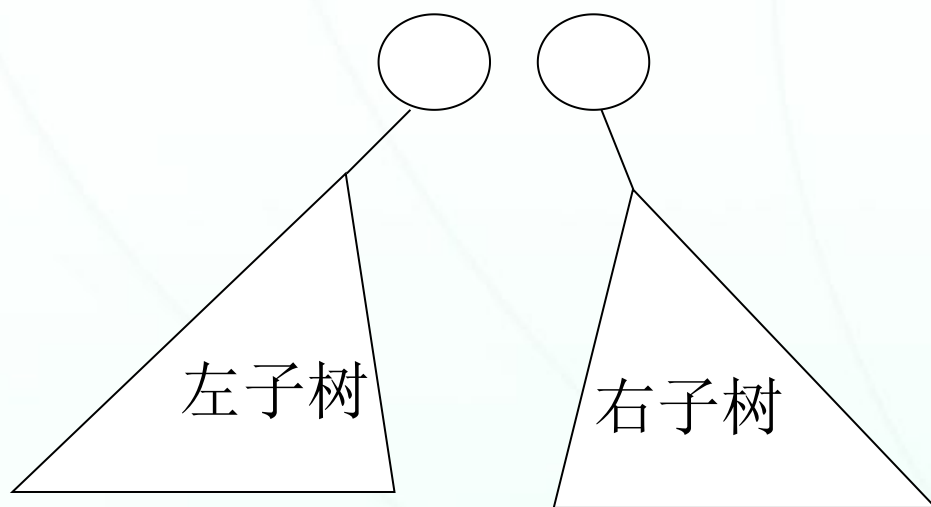
二叉树有五种基本形态：



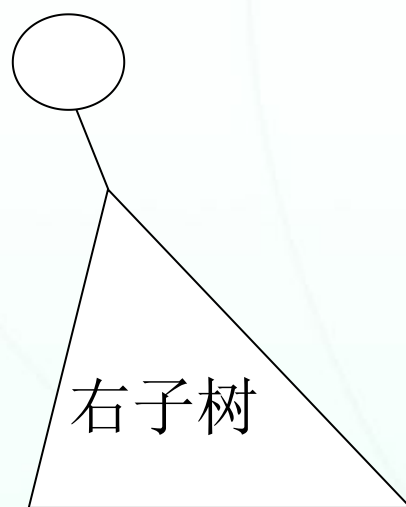
二叉树的基本形态:



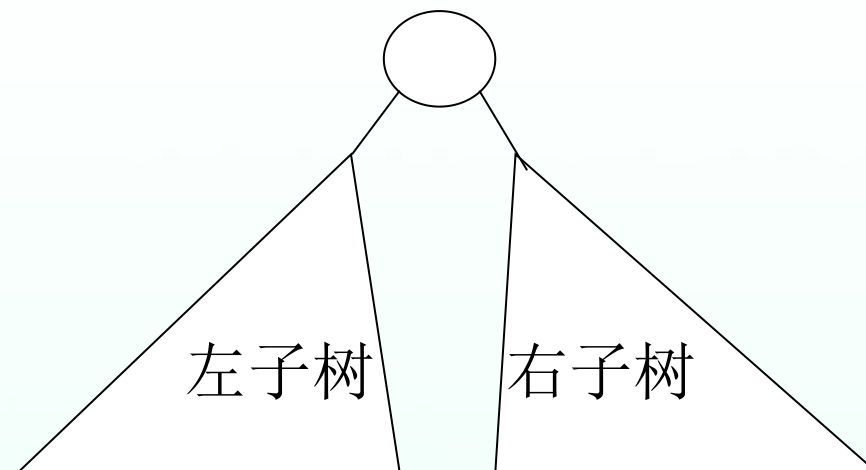
(1) 空二叉树 (2) 只有一个根结点



(3) 有根结点和左子树



(4) 有根结点和右子树



(5) 有根结点和左, 右子树

二、二叉树的性质

1、二叉树

性质1:

第 i 层上最多有 2^{i-1} 个结点。

性质2:

深度为 K 的二叉树最多有 2^K-1 个结点。

性质3:

设叶子（度为0）数为 n_0 ，度为2的结点数为 n_2 ，则 $n_0=n_2+1$ 。



2、满二叉树与完全二叉树

满二叉树：

深度为 k ，且有 2^k-1 个结点。（对满二叉树中的结点约定编号：自根起，从上到下，从左到右）

完全二叉树：

仅允许最下层右侧分支不满，若按从上到下，从左到右为树中结点编号，则与满二叉树相同。

关系：

满二叉树是完全二叉树。

完全二叉树不一定是满二叉树。



3、完全二叉树性质

性质4:

n 个结点，深度为 K 的完全二叉树，其

$$K = \lfloor \log_2 n \rfloor + 1$$

性质5:

对一棵 n 个结点深度为 K 的完全二叉树上的结点按层次编码（从第一层到第 K 层，从左到右），则树中编号为 i 的结点（ $1 \leq i \leq n$ ）有：

a: 当 $i=1$ 时，此点是根；

b: 当 $i>1$ 时， i 的双亲是 $\lfloor i/2 \rfloor$ ；

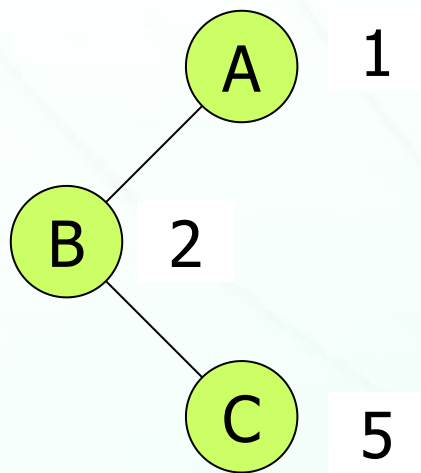
c: 当 $2i>n$ 时， i 是叶点（无左孩子）；当 $2i \leq n$ 时， $2i$ 是 i 的左孩子；

d: 当 $2i+1>n$ 时， i 结点无右孩子；当 $2i+1 \leq n$ 时， $2i+1$ 是 i 的右孩子。

三、二叉树存储结构

1、顺序存储结构

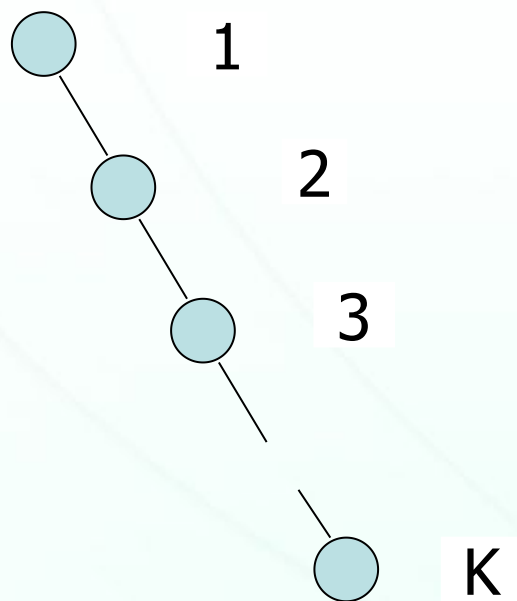
用一维数组A作为二叉树的存储结构， $A[i]$ 表示编号为i的结点。



0	1	2	3	4	...
A	B			C	

特点：浪费存储空间

特例:

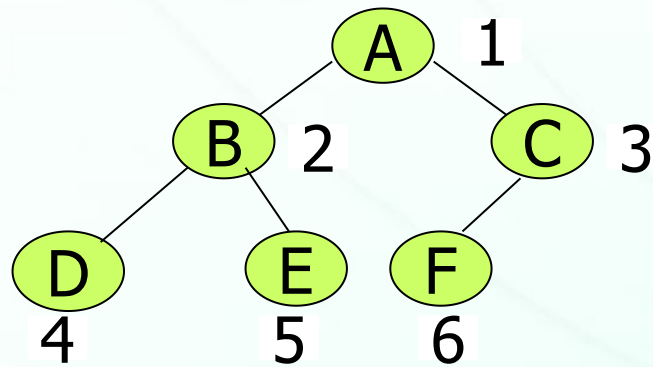


深度为K，只有K个结点的单支树需要 2^K-1 个存储单元。（由二叉树的性质2知，深度为K的二叉树至多有 2^K-1 个结点）。而真正有用的只有K个空间，其它绝大多数空间是空闲的。造成了存储空间的极大浪费。

顺序存储结构适用于完全二叉树:

特点:

即不浪费空间, 又可以快速确定其结点的位置; 对已知的结点 i , 其左孩子为 $2i$, 右孩子为 $2i+1$, 双亲为 $\lfloor i/2 \rfloor$ 。



完全二叉树

A

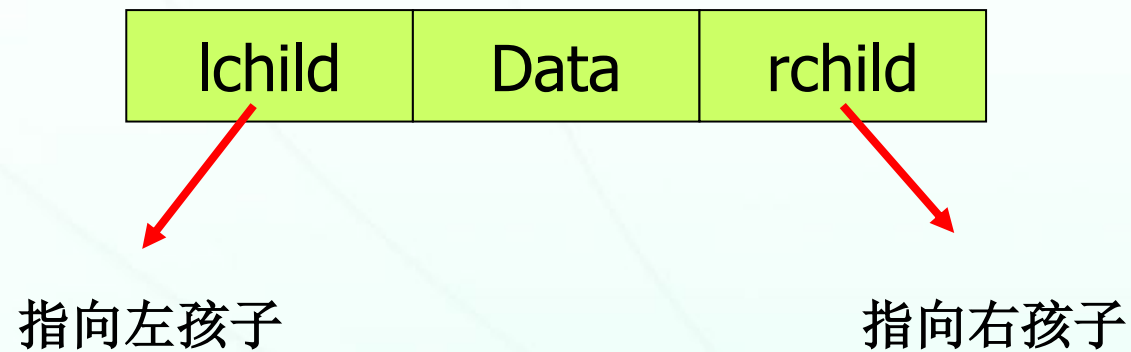
0	1	2	3	4	6
A	B	C	D	E	F

结点在存储空间中的相对位置
恰好表现出了完全二叉树中结
点之间的关系。

2、链式存储结构

二叉链表:

链表中每个结点至少含有三个域：数据域、左指针域和右指针域。



§ 3. 二叉树的遍历

一、遍历二叉树

遍历：

按一定次序访问二叉树上每个结点恰一次；

访问：

访问代表一次操作；

访问次序：

先左后右。并以访问根的次序不同分为不同的三种方式：先序、中序、后序。



先序:

先访问根结点，然后访问左子树，再访问右子树。

中序:

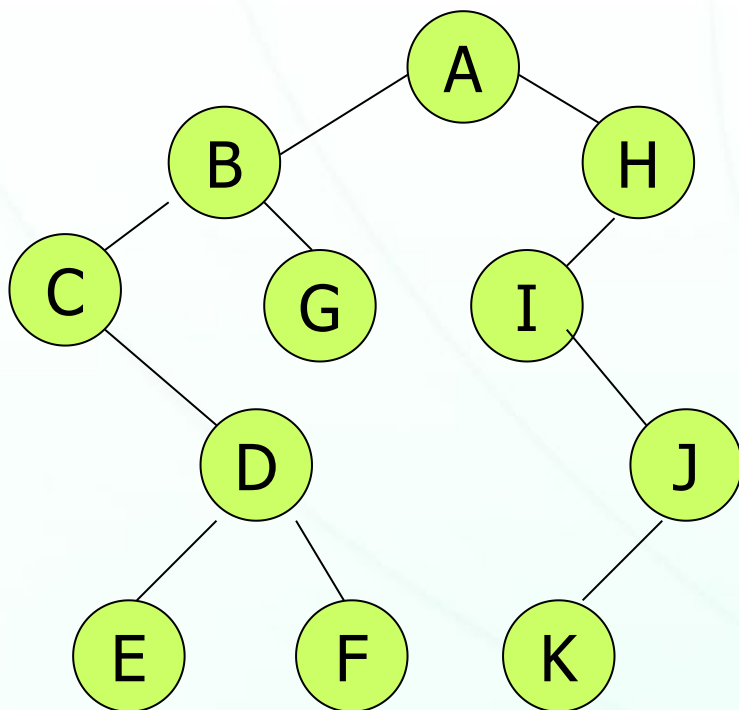
先访问左子树，然后访问根结点，再访问右子树。

后序:

先访问左子树，然后访问右子树，再访问根结点。



例1:



先序: ABCDEFGHIJK

中序: CEDFBGAIKJH

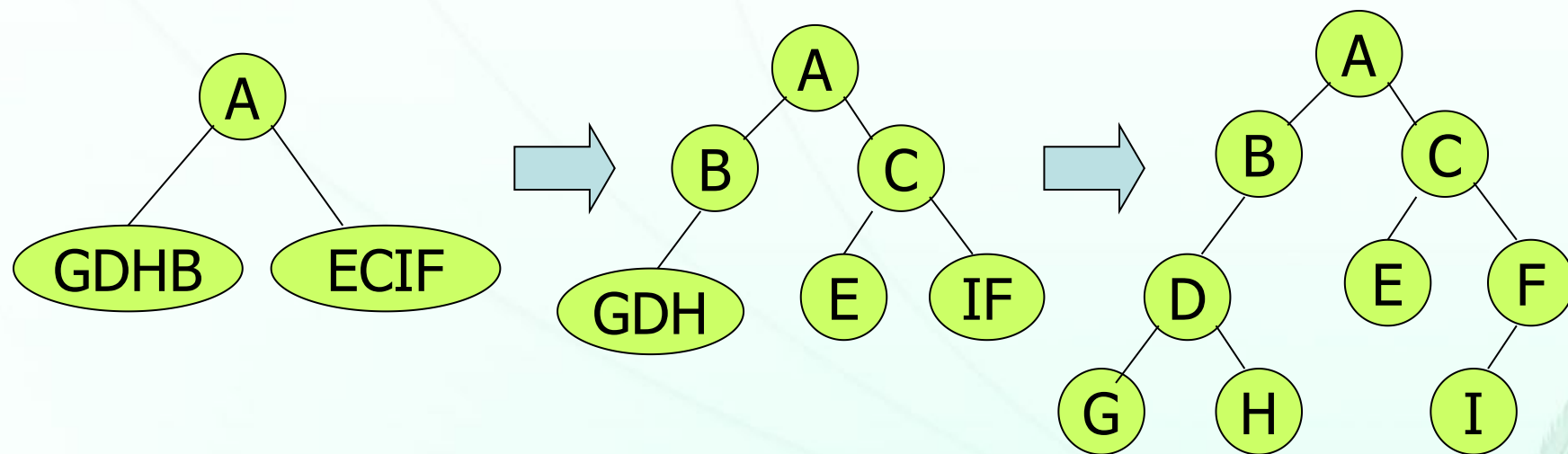
后序: EFD CGBKJIHA

例2：已知先序、中序可以惟一确定一棵二叉树

已知：

中序：GDHBAECIF

先序：ABDGHCEFI

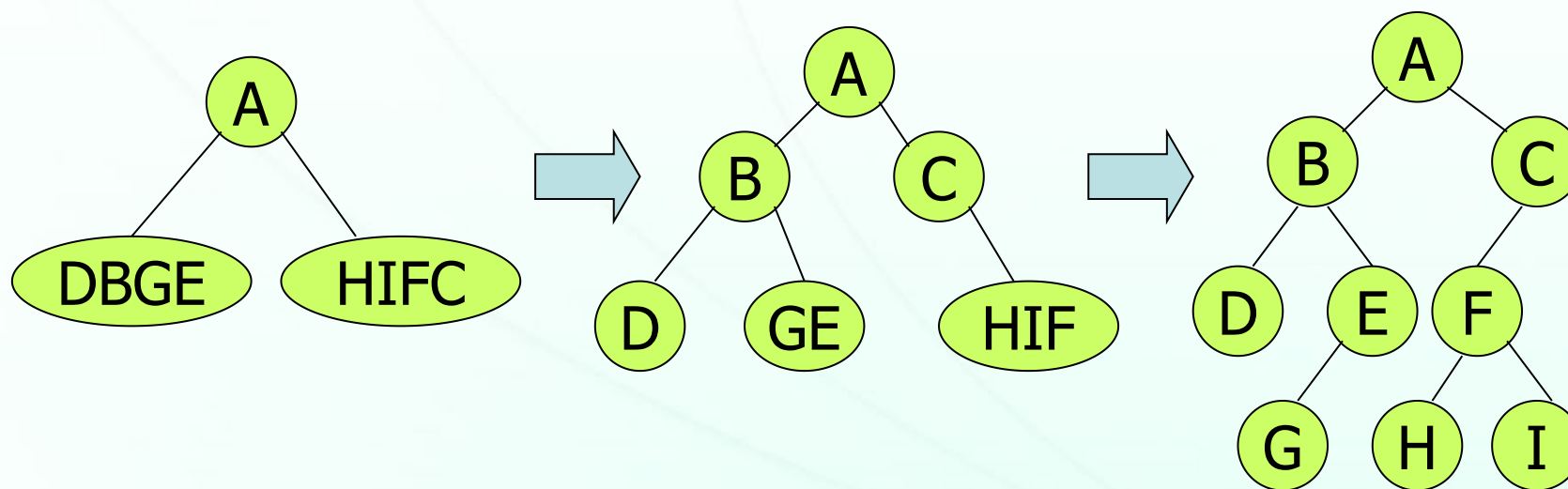


例3：已知中序、后序可以惟一确定一棵二叉树

已知：

中序：DBGEAHFIC

后序：DGEBHIFCA



例4：已知先序、后序不能惟一确定一棵二叉树

已知：

先序：A B

后序：B A



两棵不同的二叉树

以中序为例讨论二叉树的递归遍历算法：

中序遍历二叉树的递归算法分析：

中序：访问左子树 $\longrightarrow$ 访问根 $\longrightarrow$ 访问右子树

基值： $bt = \text{NULL}$ (bt 为二叉树的根结点指针)。当二叉树为空时。

归纳项：即是对子树（子树仍为二叉树）的遍历操作。

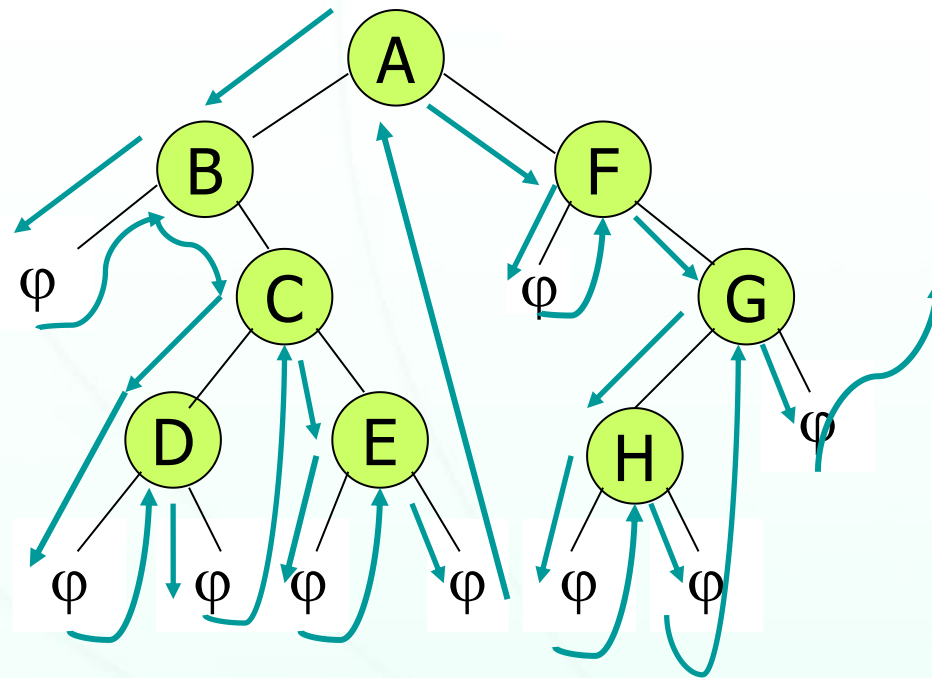
算法思想：

若二叉树为空，则遍历结束；否则

遍历左子树
访问根结点
遍历右子树



中序递归算法执行过程的简化展开图：



每深入一层代表一次调用；退上一层代表一次返回；从左子树返回仅一层；接着访问根结点；从右子树返回，表明当前层遍历结束，可以连续返回。

二、线索二叉树

1、什么是二叉树的线索化？

非线性结构的二叉树经过遍历（先序、中序、后序），结点排成线性有序（前趋、后继惟一）。但二叉树本身不具有这种信息，访问后即消失；为了保持在遍历过程中得到的前趋、后继信息，则需边遍历，边建立线索。

线索：即是指向前趋、后继结点的指针。

线索二叉树：加上线索的二叉树称为线索二叉树。

线索化：对二叉树进行遍历使其成为线索二叉树的过程称为二叉树的线索化。

更多优质自考资料尽在百度贴吧自考乐园俱乐部
(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



2、怎样线索二叉树

(1) 增加两个标志位

每个标志位仅占1个存储位(bit)

结点结构:

lchild	LTag	data	RTag	rchild
--------	------	------	------	--------

若LTag=0(Link), 则lchild指向左孩子;

RTag=0(Link), 则rchild指向右孩子;

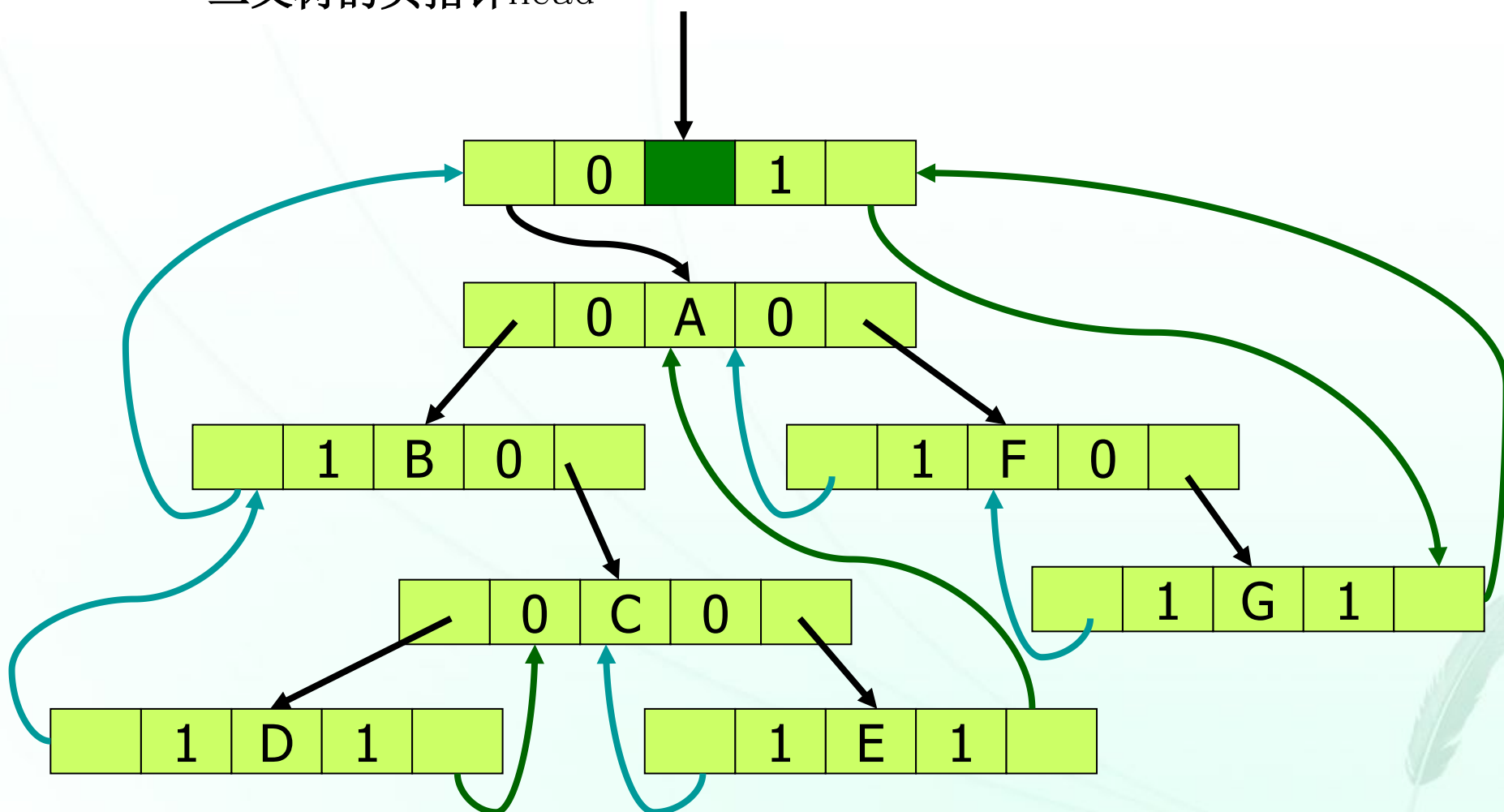
若LTag=1(Thread), 则lchild指向线索——前趋;

RTag=1(Thread), 则rchild指向线索——后继。



带头结点的中序线索链表:

二叉树的头指针Head

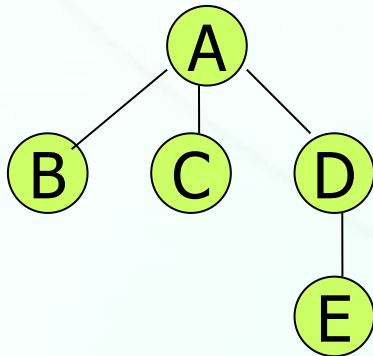


§ 4. 树和森林的存储与运算

一、树的存储表示

1、双亲表示法（顺序存储结构）

开辟一组连续空间存储树的结点，每个结点中附设一个指示双亲结点的双亲域。



Data:
Parent:

0	1	2	3	4
A	B	C	D	E
-1	0	0	0	3

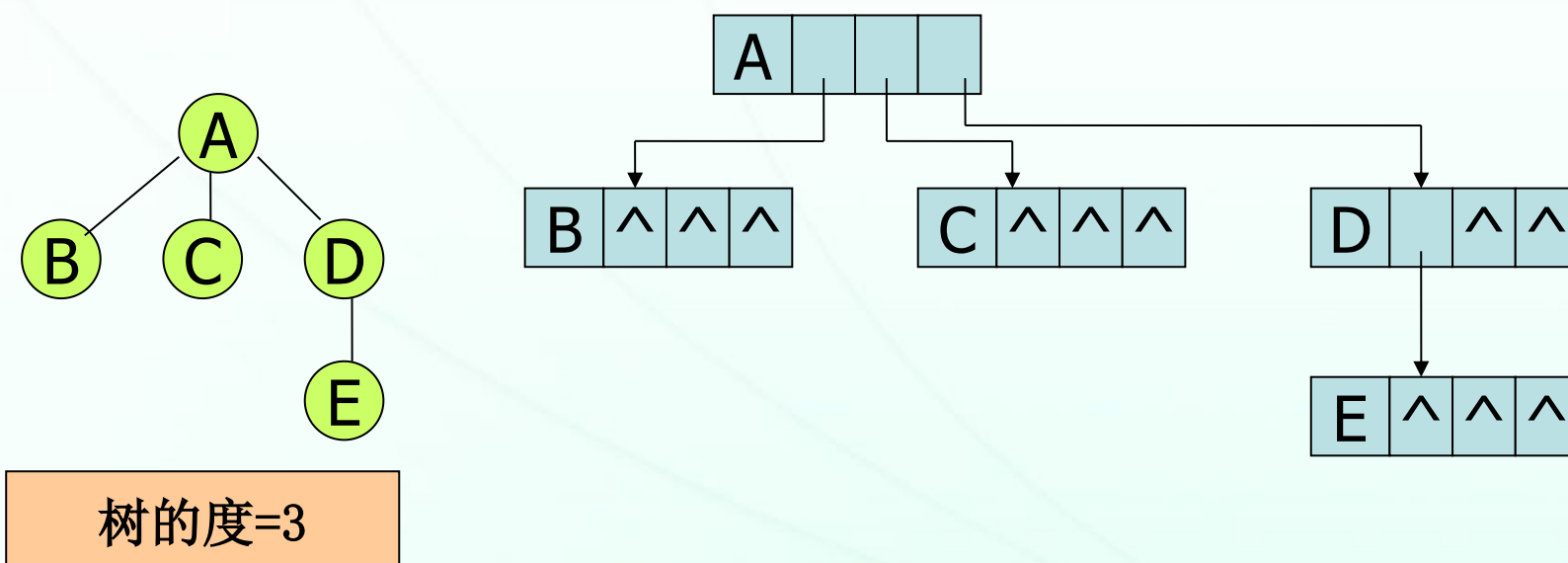
特点：找结点双亲容易，但找孩子不容易

2、孩子表示法（链式存储结构）

(1) 按树的度设结点指针域

特点：

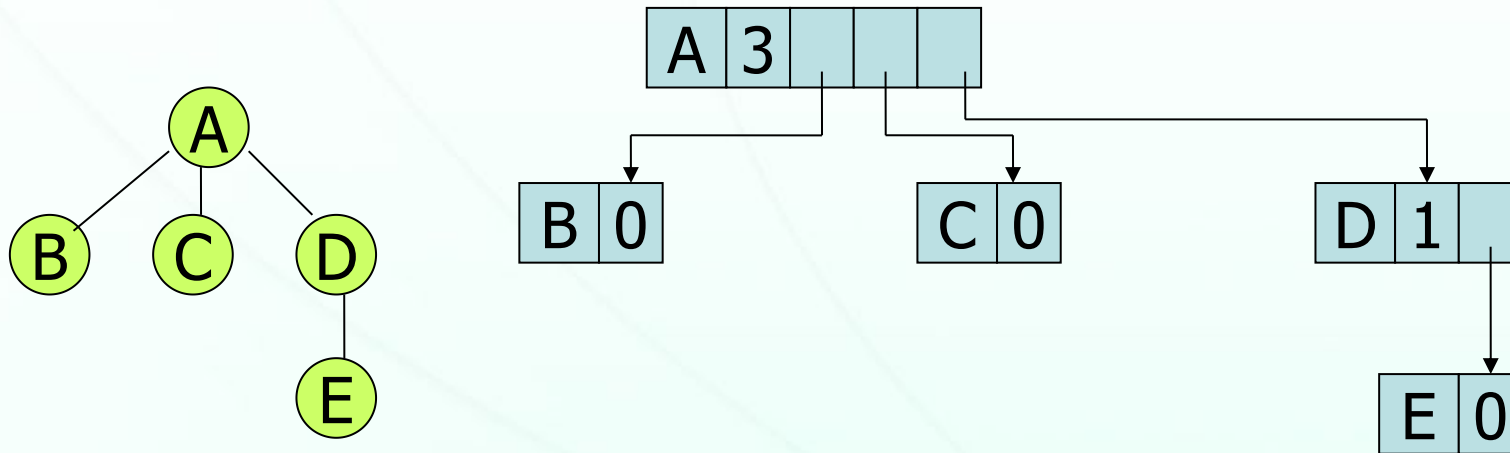
每个结点的结构一致，操作简单，但浪费存储空间。



(2) 按结点的度设结点指针域

特点:

每个结点的结构不一致，操作复杂，但节省存储空间。



(3) 按孩子结点排成线性表

特点:

节省空间,

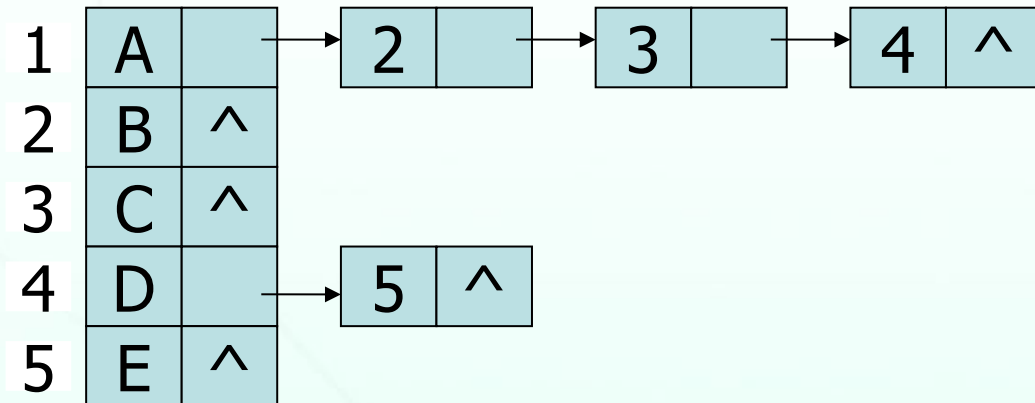
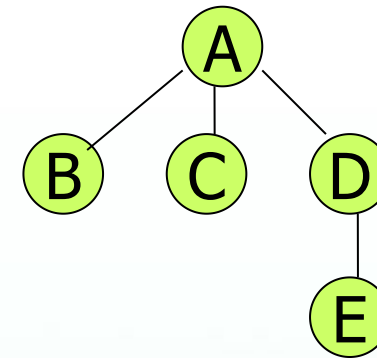
操作也较容易;

问题:

这种结构不能

反映出树的层

次性特征。



每个结点的孩子链表

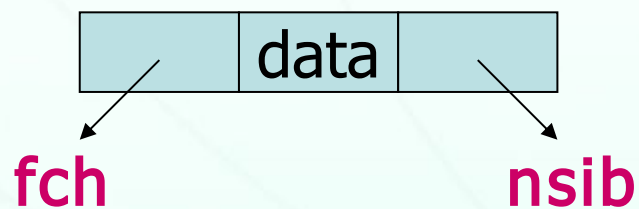
3、孩子兄弟表示法（二叉树表示法）

在这种表示法中，树中每个结点有三个域：

数据域：存放结点数据

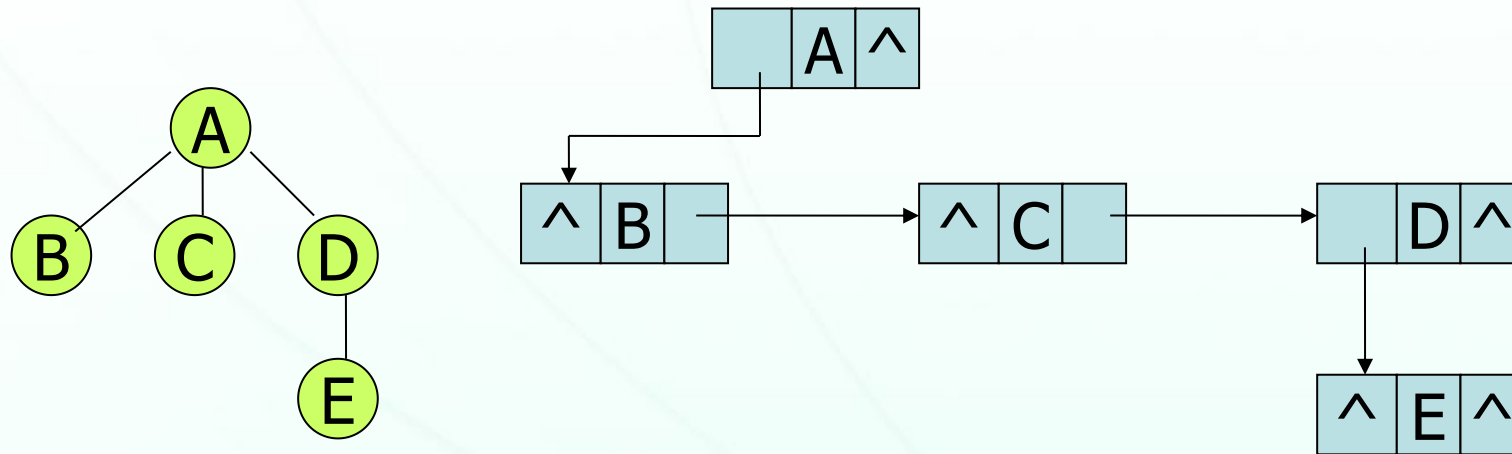
左指针域：指向该结点的第一个孩子结点

右指针域：指向该结点的右兄弟结点



特点:

便于实现树的各种操作, 节省空间, 并且能够描述树的层次结构。



二、森林与二叉树的转换

二叉树和树都可以用二叉链表作为其存储结构。因此，以二叉链表作为中间媒介，可以导出树与二叉树之间存在对应关系。

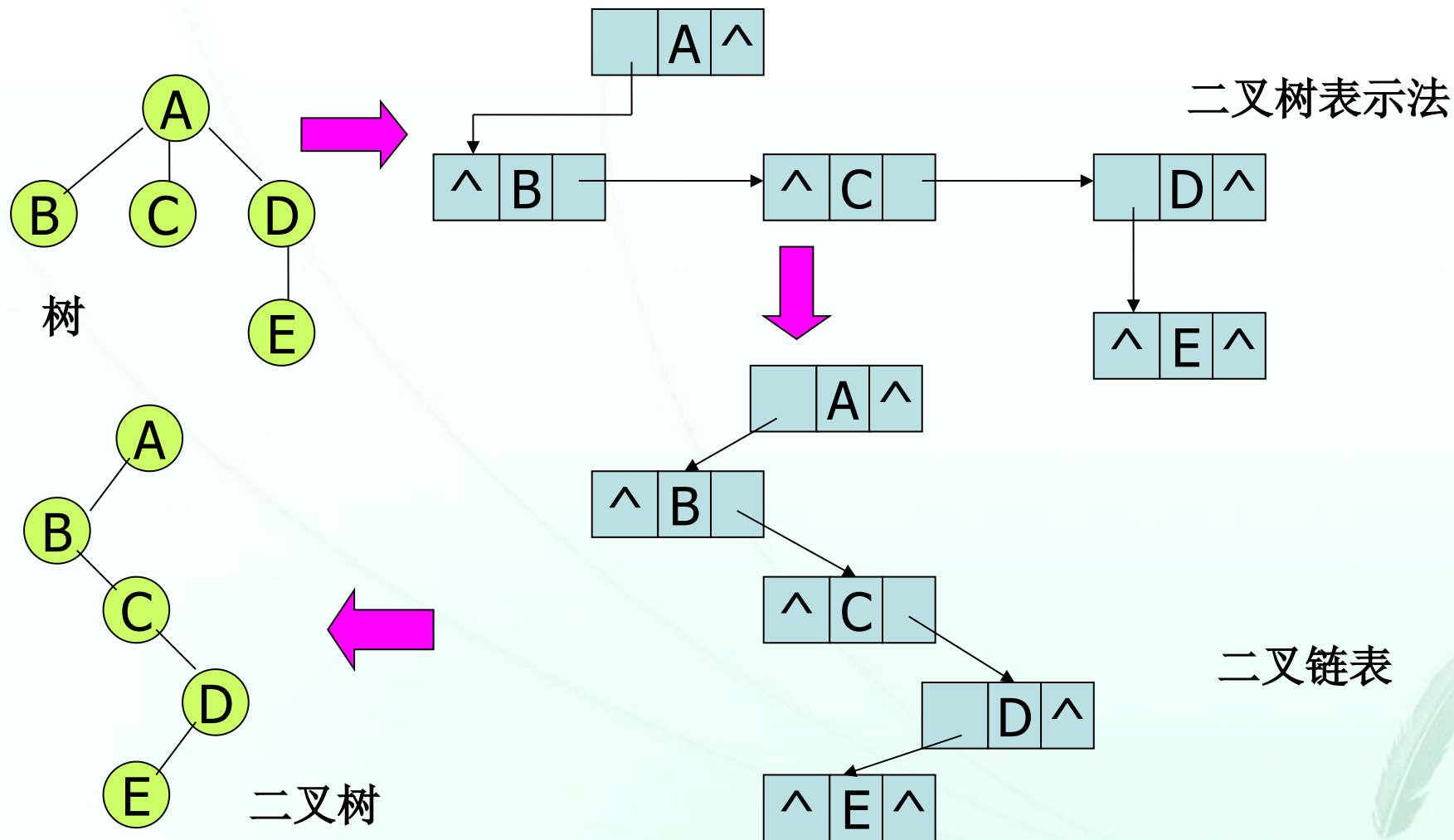
一棵树与惟一的一棵二叉树一一对应。

更多优质自考资料尽在百度贴吧自考乐园俱乐部
(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



举例：



1、树转换成二叉树

保持双亲和第一个孩子连线，其他连线去掉；

兄弟之间连线；

使右兄弟结点成为右孩子，使第一个孩子成为左孩子。



2、森林转换为二叉树

将每棵树转化为二叉树；

将各棵二叉树的根相连；

使二叉树的根成为右孩子。



3、二叉树转换成森林

若某结点是其父结点的左孩子，则把该结点的右孩子、右孩子的右孩子、.....，都与该结点的父结点用线连；

去掉树中所有父结点到其右孩子的连线，生成森林。



三、树与森林遍历

1、树的遍历

遍历规则：

(1) 先根遍历：

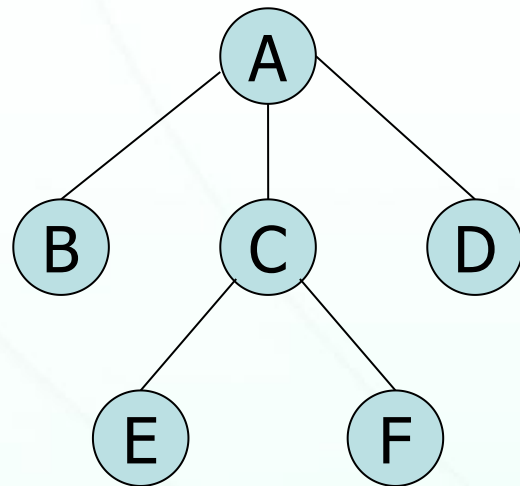
先访问树根，再依次先根遍历根的每棵子树。

(2) 后根遍历：

先依次后根遍历根的每棵子树，再访问树根。

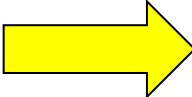


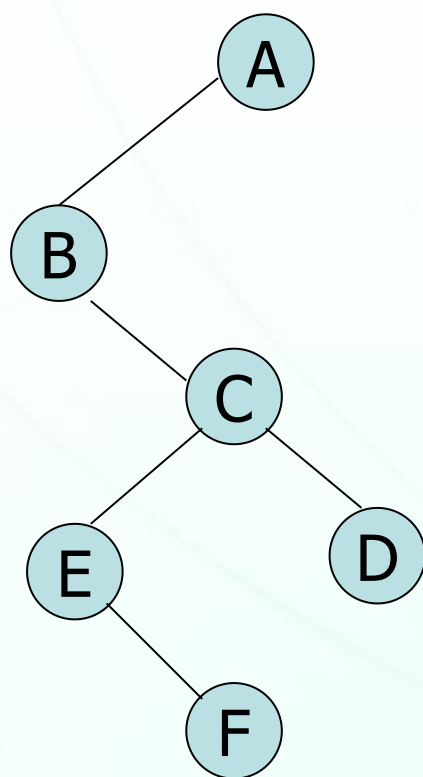
例：



先根：ABCEFD

后根：BEFCDA

树  二叉树 (续上例)



先序: ABCEFD
中序: BEFCDA

结论：（续上例）

树的**先根**遍历相当于其转化的二叉树的**先序**遍历；

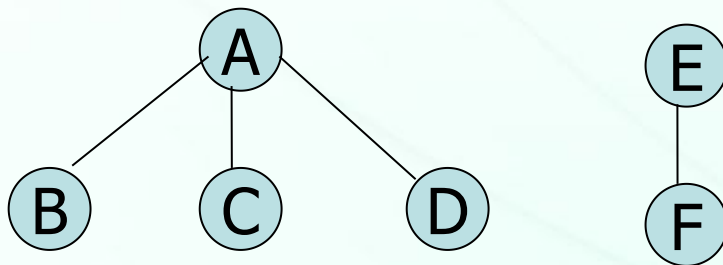
树的**后根**遍历相当于其转化的二叉树的**中序**遍历；



2、森林的遍历

(1) 先序(先根)遍历森林

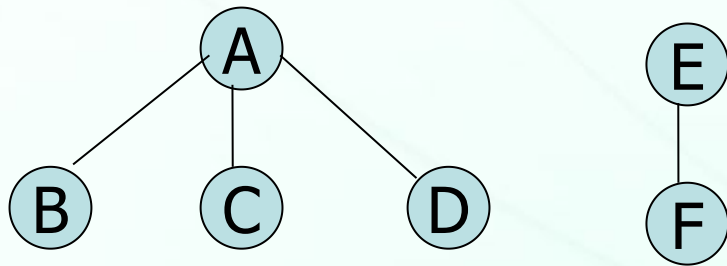
- A. 访问第一棵树的树根;
- B. 先序遍历第一棵树根的子树森林;
- C. 先序遍历除第一棵树之后, 剩余的树构成的森林。



先序: ABCDEF

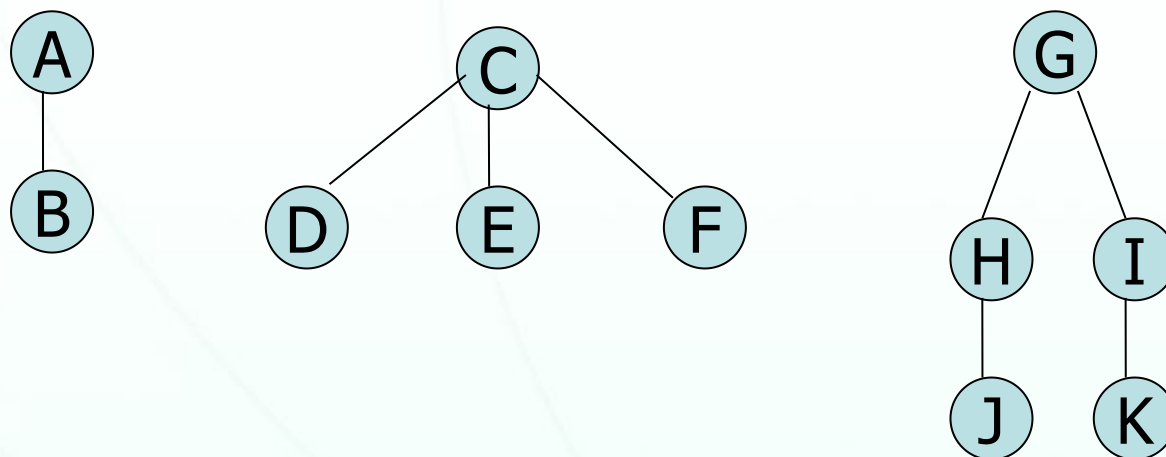
(2) 中序(后根)遍历森林

- A. 中序遍历森林中第一棵树的根结点的子树森林;
- B. 访问第一棵树的根结点;
- C. 中序遍历除第一棵树之后剩余的树构成的森林。



中序: BCDAFE

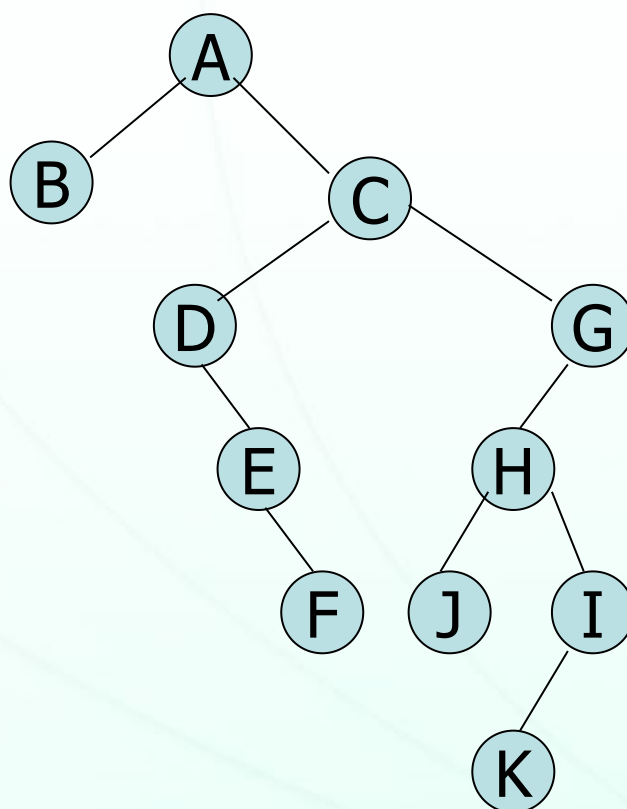
例:



先根: ABCDEFGHJIK (二叉树的先序)

后根: BADEFCHJKIG (二叉树的中序)

森林转化的二叉树:



§ 5. 哈夫曼树及其应用

一、概念和术语

1、路径

2、路径长度

3、结点的权

4、结点的带权路径长度

5、树的带权路径长度

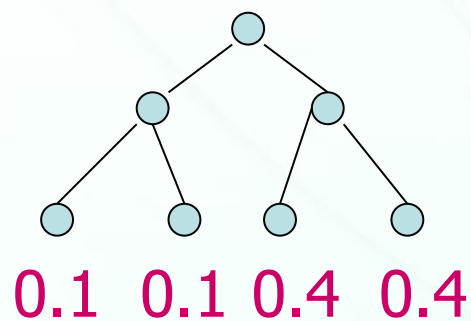
6、哈夫曼树



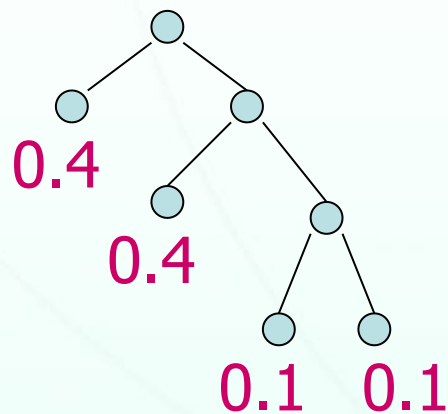
(续三)

例:

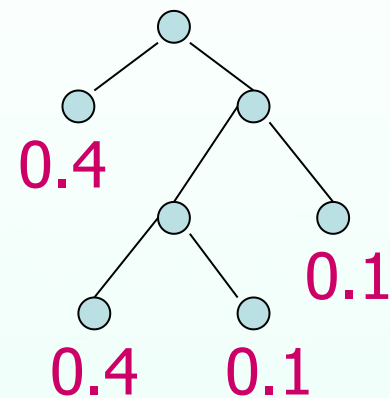
给定四个叶子结点a、b、c、d，分别带权0.1、0.1、0.4、0.4，由它们构成不同的带权二叉树，分别求WPL。



WPL=2



WPL=1.8



WPL=2.1

二、哈夫曼树的构造

哈夫曼树构造算法：(P91)

根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 构成 n 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$ ，其中每棵二叉树 T_i 中只有一个带权为 w_i 的根结点，其左右子树为空。

在 F 中选取两棵根结点的权值最小的树作为左右子树构造一棵新的二叉树，且置新的二叉树的根结点的权值为左右子树上根结点的权值之和。

在 F 中删除这两棵树，同时将新得到的二叉树加入 F 中。

重复（2）和（3），直到 F 只含一棵树为止。这棵树便是哈夫曼树。

例：给定权值 $\{7, 5, 2, 4\}$ ，构造哈夫曼树。



前缀码

给定一个码的集合序列，若没有一个序列是另一个序列的前缀，则称这个集合中的码为**前缀码**。

例：

{000, 001, 01, 10, 11} ——前缀码

{0, 1, 10, 11} ——不是前缀码

特点：

前缀码是码长不等的编码，既可以缩短报文的长度，又容易翻译。

（本章结束）



第七章 图

§ 1 . 基本概念与术语

一、图的定义

定义:

简单地说, 图是由一些结点和连接两个结点之间的边组成。



§ 2. 图的存储表示和图的生成

一、图的存储表示

1、邻接矩阵表示法（数组表示法）

用一维数组存储图中顶点的信息；

用矩阵（二维数组）表示图中各顶点之间的相邻关系。

图的顺序存储结构

(1) 邻接矩阵的定义

图 $G=(V, E)$, n 个顶点, 邻接矩阵用 $A[1..n, 1..n]$ 存放:

$$A[i][j] = \begin{cases} 1 & \text{若 } \langle v_i, v_j \rangle \text{ 或 } (v_i, v_j) \in E, i \neq j \\ 0 & \text{否则} \end{cases}$$

对带权的图:

$$A[i][j] = \begin{cases} w_{ij} & \text{若 } \langle v_i, v_j \rangle \text{ 或 } (v_i, v_j) \in E, i \neq j, \text{ 且权为 } w_{ij} \\ 0 & \text{否则} \end{cases}$$



(2) 对称性

无向图:

邻接矩阵对称, 只需存入下三角 (或上三角),

所用空间为 $n(n+1)/2$

有向图:

不一定对称, 需 n^2 个存储单元。

更多优质自考资料尽在百度贴吧自考乐园俱乐部

(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



2、图的邻接表（图的一种链式存储结构）

(1) 无向图的邻接表

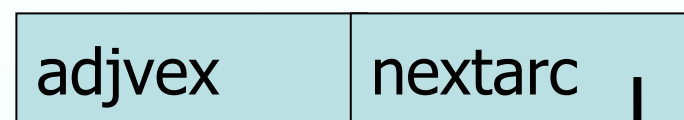
头结点：



顶点

指向与该顶点
邻接的第一个
邻接点

表结点：



邻接点在
图中的
位置

指向下一个
邻接点

构造方法:

与同一顶点邻接的所有邻接点构成一个单链表，用表结点描述；

各链表设置表头结点，由头结点描述；

各表头结点构成一个顺序表。



(2) 有向图的邻接表及其逆邻接表

邻接表（出度表）：

结点结构同无向图。

以 v_i 为顶点的邻接点单链表上的结点都是以 v_i 为**弧尾**的邻接点。

顺序表部分定义同无向图的邻接表。

逆邻接表（入度表）：

结点结构同无向图。

以 v_i 为顶点的邻接点单链表上的结点都是以 v_i 为**弧头**的邻接点。

顺序表部分定义同无向图的邻接表。



§ 3. 图的遍历

遍历:

按一定次序，由图的某一顶点出发，沿着图的边访问所有顶点恰一次。

设置访问标志数组: `visited[i]`

$[1 \leq i \leq n]$ (n 为图中顶点个数)

False 表示未被访问

True 表示已被访问



一、深度优先搜索（DFS）

搜索策略：

访问当前顶点 v_i ，寻找与 v_i 相邻且未被访问顶点 v_j ，若存在，将其作为当前点，访问，并重复上述搜寻过程。

否则（所有邻接点都被访问过），退回一步，寻找与前一个顶点相邻的未被访问顶点，访问，并重复上述过程。

若上述过程无法访问图中的所有顶点，则另选一个新顶点重新开始。

深度优先搜索是一种**纵向搜索**的过程。



二、广度优先搜索（BFS）

搜索策略：

访问出发点 v_0 ，依次访问 v_0 的各个未曾访问过的邻接点，然后分别从这些邻接点出发重复上述过程。

若上述过程无法访问图中的所有顶点，则另选一个新顶点重新开始。

广度优先搜索是一种横向搜索的过程。

使用队列结构。



算法思想

从指定顶点出发，每访问一个顶点就把它排在队尾，然后从队头取出一个顶点，访问该顶点所有未被访问的邻接点，如此，直到队空。

若图中仍有未被访问的顶点，则另选一个顶点为出发点，重复上述过程。



§ 4. 生成树&最小生成树

一、生成树

1、**定义**：一个**连通图** G 的子图如果是一棵包含 G 的所有顶点的树，则该子图称为 G 的**生成树**。

2、**算法思想**：DFS或BFS生成树。



二、最小生成树

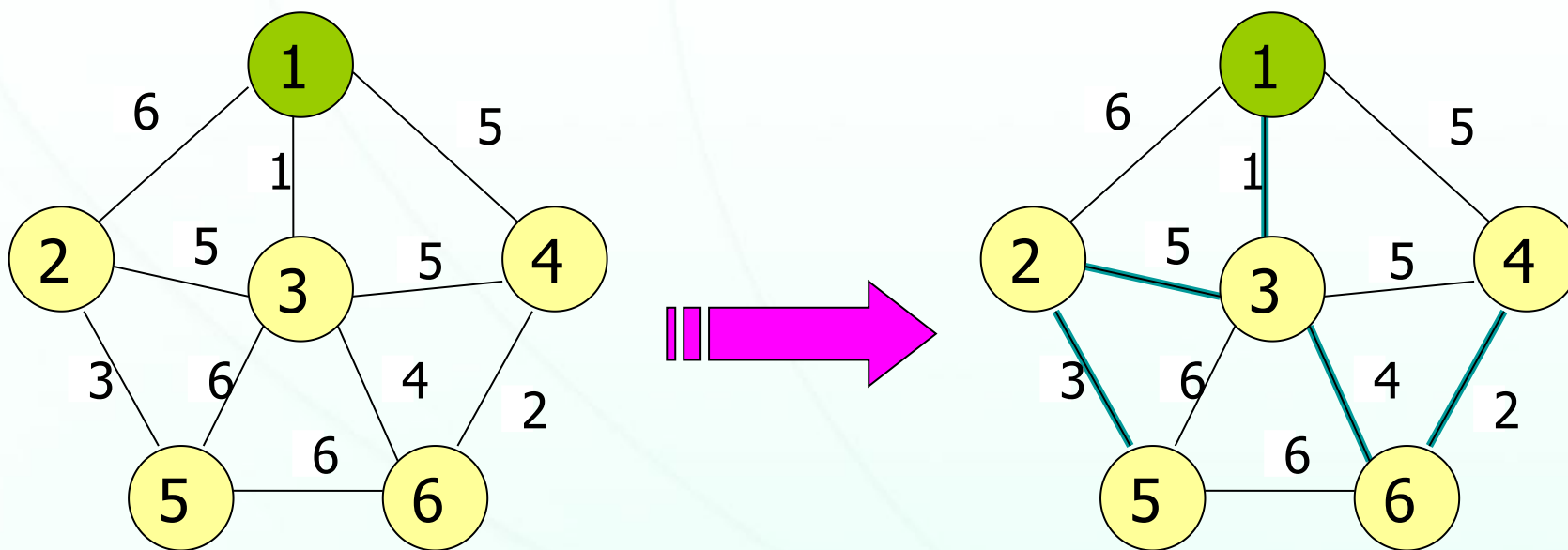
1、定义

连通网 $G=(V, E)$ ，边是带权的，因而 G 的生成树的各边也是带权的。将生成树的各边的权值总和称为生成树的权，并把**权最小**的生成树称为 G 的**最小生成树**。



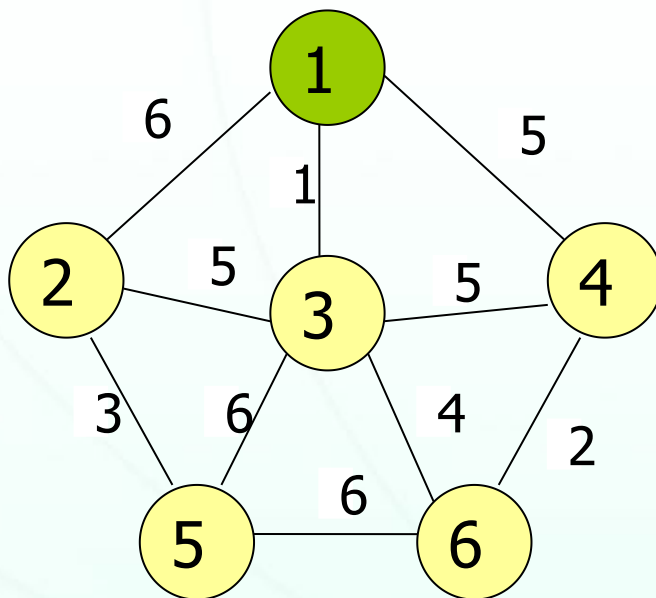
例：用Prim算法求最小生成树

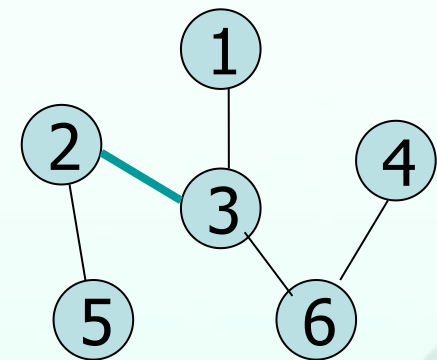
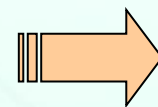
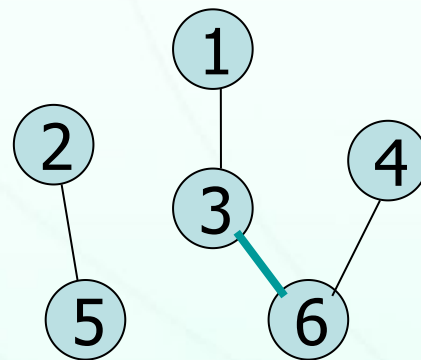
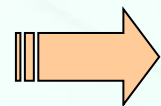
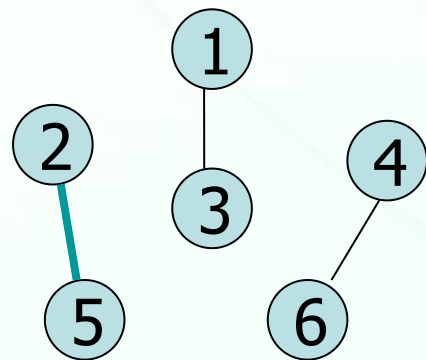
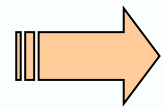
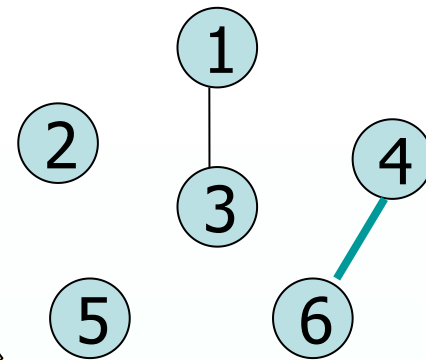
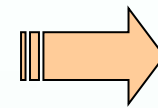
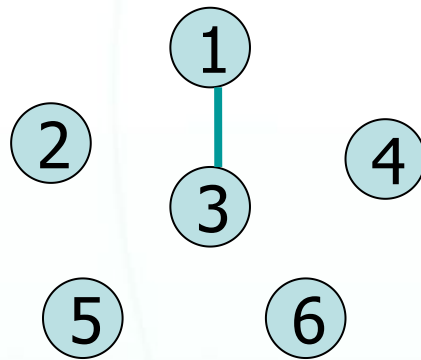
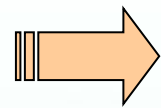
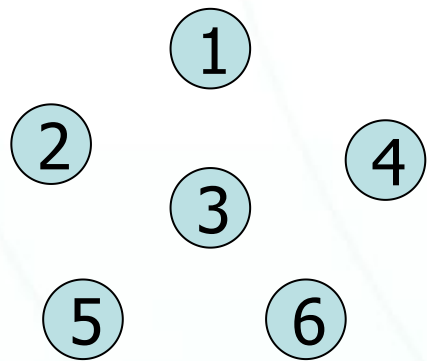
连通图如下所示：



例：用克鲁斯卡尔算法求最小生成树

连通图如下所示：



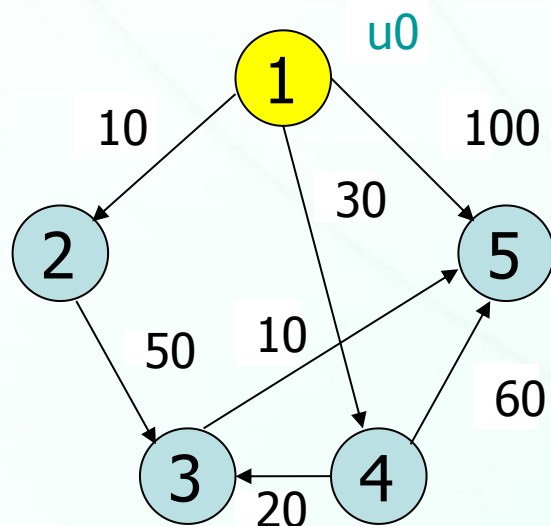


§ 5. 最短路径

一、单源最短路径问题

单源最短路径问题

给定带权有向图，求从指定源点到图中其余各点的最短路径。



u0到其余各顶点的最短路径

源点	中间顶点	终点	路径长度
1		2	10
1		4	30
1	4	3	50
1	4,3	5	60

§ 6. 拓扑排序

将集合中偏序关系变为全序关系称为拓扑排序。



一、拓扑排序

1、相关概念

AOV网：顶点表示活动，弧表示活动间的优先关系的有向图。

在无回路的AOV网中，所有的活动可以排成一个线性序列，使得每个活动的所有前驱活动都排在该活动的前边，称此序列为**拓扑序列**，由AOV网构成拓扑序列的过程称为**拓扑排序**。



拓扑排序算法思想:

选择有向图中入度为0的顶点输出，并从图中删去该点相邻弧；

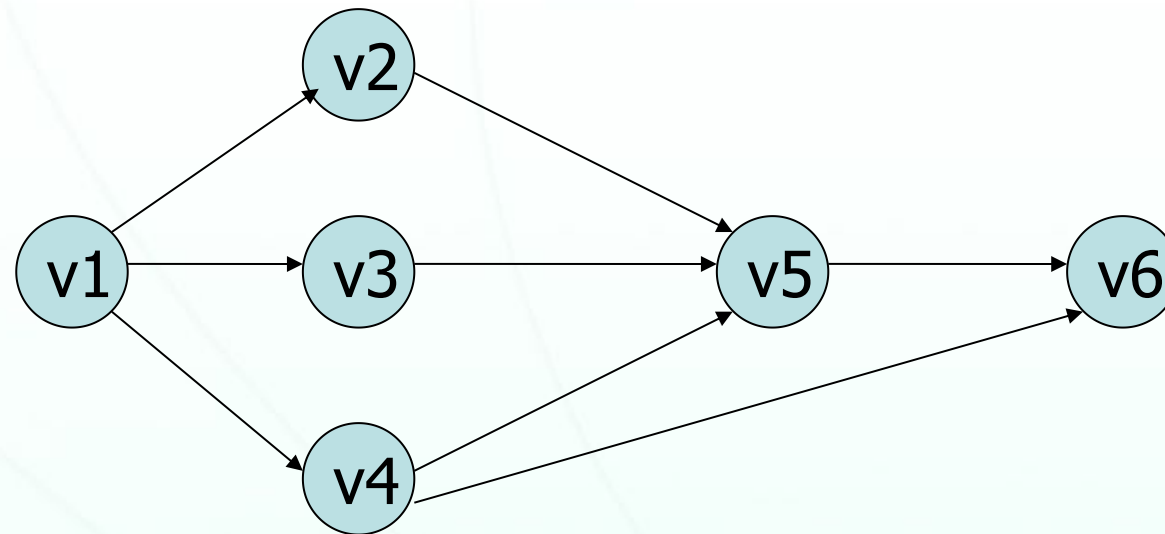
重复上述操作，直到图中全部顶点均被输出或图中已不存在入度为0的顶点（有回路）。

更多优质自考资料尽在百度贴吧自考乐园俱乐部
(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



实例：了解算法的执行过程



求精算法

时间复杂度： $T(n)=O(n+e)$

（本章结束）



第八章 内部排序

排序 (Sorting) 是将任意顺序的序列重新排成按关键字有序的序列。

排序的目的之一是提高查找效率。

排序是数据处理领域中的常用运算。



§ 2. 排序概念

趟 (Pass)

实现一个元素的有序性所完成的操作。

稳定性

若在排序前两个记录在序列中的关系为 $\dots R_i \dots R_j \dots$ ，且 R_i 与 R_j 的关键字相同 $\text{Key}_i = \text{Key}_j$ ，排序后若 $\dots R_i \dots R_j \dots$ 称排序方法稳定，若 $\dots R_j \dots R_i \dots$ 称排序方法不稳定。



内部排序的相关概念

排序方法分类

插入排序

交换排序

选择排序

归并排序

基数排序

排序算法的关键操作

比较：比较两个关键字的大小。

移动：将记录从一个位置移动至另一个位置。



§ 2. 插入排序

一、直接（简单）插入排序

基本思想：

每一趟都在已排好序的有序子表中插入一个元素，使有序子表不断扩大。

趟：

在含有 $i-1$ 个记录的有序子表中，插入第 i 个元素成为含有 i 个元素的有序子表的过程。

算法实现：

引进监视哨的作用。



算法分析

空间：增加一个监视哨

时间

比较

最好情况（正序）： $n-1$ 次

最坏情况（逆序）： $(n+2)(n-1)/2$

移动

最好（正序）： $2(n-1)$

最坏（逆序）： $(n+4)(n-1)/4$

平均时间复杂度： $O(n^2)$

稳定性：算法稳定



二、希尔排序 (Shell排序)

又称缩小增量排序，是直接插入排序的改进。

基本思想

将待排记录序列按增量 dh 值分成若干子表进行直接插入排序；然后缩小增量值，再分割，再排序，直至整个序列“基本有序”时，再对整个序列做一次直接插入排序。



算法分析

增量选取方法

增量序列不同，时间复杂度不同

稳定性：算法不稳定

希尔排序的特点



§ 3. 交换排序

一、起泡排序

基本思想

相邻元素依次进行比较，小的前移，大的后移，第一趟结束后，最大元素放在了最后；再进行第二趟，使次大元放在最大元素之前；继续，直到在一趟排序中没有交换发生，排序结束。



算法分析

最好情况（正序）

比较次数： $n-1$

移动次数： 0

最坏情况（逆序）

比较次数： $\sum_{i=1}^{n-1} (n-i) = n(n-1)/2$

移动次数： $\frac{3}{2} n(n-1)$

时间复杂度： $O(n^2)$

稳定性： 算法稳定



二、快速排序

基本思想

每趟让待排表中的第一元素作支点，排序后入终位 k ，将原表一分为二，使得 $r[1] \sim r[k-1]$ 中元素小于等于 $r[k]$ ，而 $r[k+1] \sim r[n]$ 中元素都大于等于 $r[k]$ ，递归进行，直到表长为1时，排序结束。



一趟快速排序算法

支点放入 x

附设两个指针 i 和 j ：初始 i 指向第一元， j 指向第 n 元

从右向左，放过 $\geq x.\text{key}$ 元（比 x 大的元素）， j 变化，当 $r[j].\text{key} < x.\text{key}$ ，放入 i 位置

从左向右，放过 $\leq x.\text{key}$ 元（比 x 小的元素）， i 变化，当 $r[i].\text{key} > x.\text{key}$ ，放入 j 位置

直到 $i=j$ ，一趟结束， i 为 x 终位



locating the pivot

[49 38 65 97 76 13 27 49']

i ↑

j ↑

[27 38 65 97 76 13 49']

i ↑

j ↑

[27 38 65 97 76 13 49']

i ↑

j ↑

[27 38 97 76 13 65 49']

i ↑

j ↑

[27 38 13 97 76 65 49']

i ↑

j ↑

[27 38 13 76 97 65 49']

i ↑

j ↑

[27 38 13 49 76 97 65 49']

i ↑ j ↑



算法分析

时间效率

最坏情况:

当待排序的记录序列已经有序时，出现恶化趋势， $T_{\text{worst}}(n) = O(n^2)$

最好情况：每次划分的两个子表的长度大致相等，支点入“中值”位， $T_{\text{best}}(n) = O(n \log_2 n)$

平均情况：数据去无序， $T(n) = T_{\text{pass}}(n) + T(k-1) + T(n-k) = O(n \ln n)$

空间效率

稳定性：不稳定



§ 4. 选择排序

一、简单选择排序

基本思想

每趟通过 $n-i$ 次关键字之间的比较, 从 $n-i+1$ 个记录中选出关键字最小的记录, 并和第 i ($1 \leq i \leq n$) 个记录交换之。

问题

简单选择排序与起泡排序在排序过程都须经过比较、交换而实现的, 二者的有何区别?



算法分析

时间效率

比较次数：（无论正序、逆序、其他任意顺序）

$$\sum_{i=1}^{n-1} (n-i) = n(n-1)/2$$

移动次数：

最好情况（正序）：0次

最坏情况（非逆序，但每趟排序过程中都有交换发生）：3(n-1)

平均时间复杂度：O(n²)

空间效率：供交换用一个存储单元的辅助空间

稳定性：不稳定



二、堆排序

堆排序的算法思想

1、建堆

将给定的原始待排序列建成一个初始堆。即先建成一棵完全二叉树，再从下往上筛选为堆。

首先由序列的第 $\lfloor n/2 \rfloor$ 结点开始与其孩子比，大的上小的下（交换），下去后若有孩子再比较，直到叶子

然后再由序列的第 $\lfloor n/2 \rfloor - 1$ 结点起，重复上述过程，直到待排序列中的第一个结点。

建成**逆堆**。得到待排序列中的最大元（根结点），并输出之。

2、根结点与最后一片叶子**交换**。

3、若不是堆，则从根开始自上而下恢复成堆。

4、重复2、3步，直到剩下一个结点为止，排序结束。



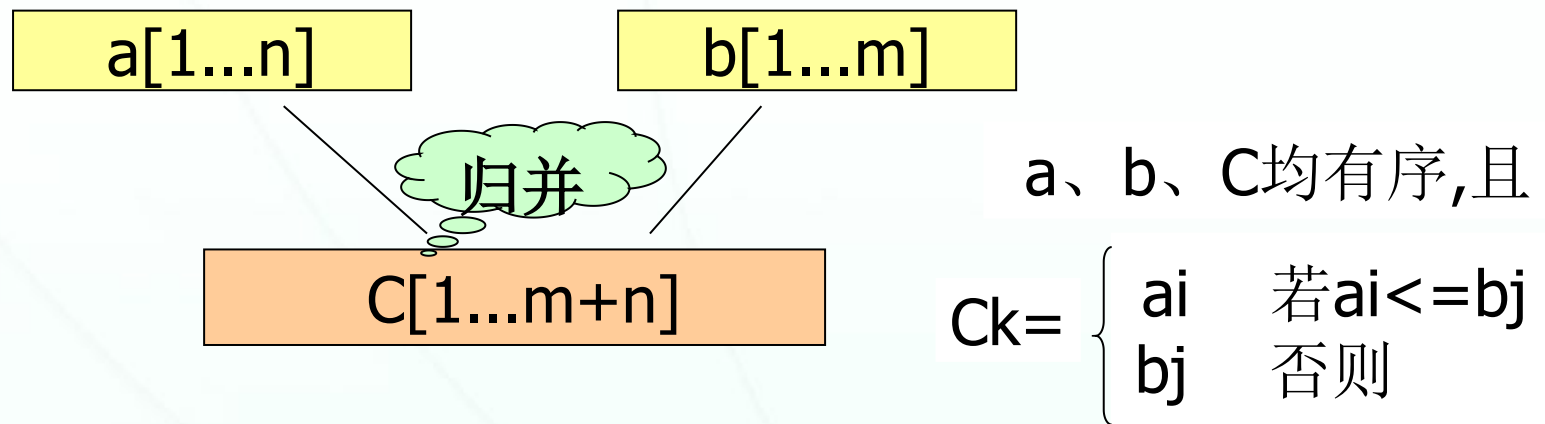
§ 5. 归并排序

归并排序是另一类不同的排序方法。“归并”的含义是将两个或两个以上的有序表合成一个新的有序表。实现算法即可采用顺序存储结构，也可采用链式存储结构。



1、归并概念

将两个或两个以上的有序表合并成整体有序的大表。



归并分类

参加归并的有序表个数为2, 称为2-路归并

参加归并的有序表个数为3, 称为3-路归并

参加归并的有序表个数 >3 , 称为多路归并

2、2-路归并的基本思想

基本思想

对于 n 个元素的待排关键字序列，先分成长度为1的 n 个有序子表，两两归并，得到 $\lceil n/2 \rceil$ 个长度为2或1的有序子表；再对这 $\lceil n/2 \rceil$ 个有序子表进行两两归并；如此重复，直至得到一个长度为 n 的有序表。

更多优质自考资料尽在百度贴吧自考乐园俱乐部

(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



3、算法分析

时间效率

时间=一趟归并时间*归并趟数

$$=O(n) * \log_2 n = O(n \log_2 n)$$

空间效率：使用了一个辅助数组 $O(n)$

算法稳定性：稳定



§ 6. 基数排序

基数排序借助多关键字排序的思想对单逻辑关键字进行排序，排序中不需要进行比较。



一、多关键字排序

设 n 个记录 $\{R_1, R_2, \dots, R_n\}$ ， R_i 的关键字为 $(K_i^0, K_i^1, \dots, K_i^{d-1})$ ， d 个关键字对任意记录 R_i 和 R_j ，若他们的关键字满足 $(K_i^0, K_i^1, \dots, K_i^{d-1}) < (K_j^0, K_j^1, \dots, K_j^{d-1})$ ，则称该记录序列有序。



举例

扑克牌按面值、花色排序。

多关键字排序方法

最高位优先法（MSD）：先按主关键字排序，再按次关键字排序。

最低位优先法（LSD）：先按次关键字排序，再按主关键字排序。

结论

无论MSD和LSD，都是采用若干次的“分配”与“收集”。

基数排序就是对单逻辑关键字采用“分配”与“收集”实现排序的方法。



二、基数排序

基数排序是借助“分配”与“收集”两种操作对单逻辑关键字采用进行排序的一种内部排序方法

单逻辑关键字： 可以看成由若干分关键字复合而成

设关键字 $\text{Key}_i = K_i^1 K_i^2 K_i^3 \dots K_i^l \dots K_i^d$ ，其中

K_i^l 称为分关键字； $C_0 \leq K_i^l \leq C_{rd-1} (1 \leq i \leq n, 1 \leq l \leq d)$

d: 关键字的最大位数

rd: 基（分关键字可取值范围）

（本章结束）



第九章 查找

§ 1 . 基本概念

一、对查找表的操作

- | | | |
|------|---|----|
| 1、查询 | } | 1类 |
| 2、检索 | | |
| 3、插入 | } | 2类 |
| 4、删除 | | |



二、查找表的分类

- 1、称只包含1类操作的查找为**静态查找**，对应的表叫**静态查找表**；
- 2、称既包含1类操作，又包含2类操作的查找为**动态查找**，对应的表叫**动态查找表**。



三、查找结果

查找成功：若表中存在要查找的记录。

查找失败：若表中不存在关键字等于给定值的记录。



四、衡量查找算法效率的标准——平均查找长度

平均查找长度

对于含有n个记录的查找表，查找成功时的平均查找长度为：

$$ASL = \sum P_i C_i$$

P_i ：查找第*i*个元素（记录）的概率。

C_i ：查找第*i*个元素（记录）所需进行的比较次数。



§ 2. 静态查找表

一、顺序表的查找

——顺序查找

静态查找表的组织方式。

顺序表，即采用线性表的顺序存储结构。

（也可采用线性链表组织静态查找表）

顺序查找

从顺序表的一端开始，用给定的key（关键字）逐个与各记录的关键字比较。如某个记录的关键字与给定值比较相等，则查找成功；反之，若直至顺序表的另一端，其关键字与给定值比较都不等，则表明表中没有所查记录，则查找不成功。



1、查找算法

顺序存储结构（顺序表）的顺序查找算法

采用**监视哨**的顺序表查找算法



2、算法分析

只考虑成功

设每个元素成功查找概率为 P_i ，等概下 $P_i=1/n$

设第 i 个元素成功查找时比较次数为 C_i ，

$$C_i = n - i + 1$$

$$ASL = \sum_{i=1}^n P_i C_i = 1/2 (n+1)$$



3、顺序查找的特点

优点

算法简单，应用广。

对表结构没有任何要求，即适用于顺序表，也适用于线性链表。

对表中元素的排列状态没有要求（有序、无序均可）。

缺点

平均查找长度较大，特别当 n 较大时，查找效率较低。



二、有序表查找

——折半查找或二分法查找

前提：静态查找表必须按关键字有序

折半查找

先确定待查记录所在范围，然后逐步缩小范围直到找到或找不到该记录为止。

给定Key与ST.elem[mid].key比较

若 $key > ST.elem[mid].key$ ：则在右子表中继续找

若 $key < ST.elem[mid].key$ ：则在左子表中继续找

若 $key = ST.elem[mid].key$ ：已找到



2、算法分析

判定树

查找过程可以用一棵二叉树来描述，通常称此树为判定树。

分析结论

折半查找法在查找成功时，给定值Key与查找表中关键字进行比较的比较次数最多不超过与折半查找过程相对应的二叉判定树的树深。

$$ASL=O(\log_2 n)$$



3、折半查找的特点

只适用于有序表（构造有序表有一定的消耗）；

只限于顺序存储结构，对线性链表不适用。



三、索引顺序表的查找

——分块查找

静态查找表的组织方式：索引顺序表

分块查找：又称索引顺序查找

1、查找表的组织

查找表中所有元素按关键字组成 b 块，每块 s 个元素；且块间元素有序，块内元素无序。

建立索引表：由索引项构成，每项两个域。索引表按关键字有序。

块内最大关键字
块所在地址(表中)



2、查找算法

查找过程分两步进行

通过索引表确定待查记录所在块——可用折半查找

在块中查找所需记录——可用顺序查找



§ 3. 动态查找表

静态查找表只能进行查找操作。若要在查找中不断地做插、删操作，应使用动态查找表。

特点：

查找表结构本身是在查找过程中动态生成的。

即对于给定值Key，若表中存在其关键字等于Key的记录，则查找成功，否则插入该记录。称此类查找表为**动态查找表**。



一、二叉排序树（BST）

1、定义

定义：如果一棵二叉树中，每个结点的值都大于它的左子树上所有结点的值，而小于它的右子树上所有结点的值，则称此树为**二叉排序树**。



2、二叉排序树的查找

算法思想

当二叉排序树不空时，首先将给定值key和根结点的关键字比较：

若根结点=key，查找成功

若根结点<key，则用同样方法在根结点的**右子树**上查找

若根结点>key，则用同样方法在根结点的**左子树**上查找

更多优质自考资料尽在百度贴吧自考乐园俱乐部

(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



3、二叉排序树的插入

插入原则

若二叉排序树为空，则将待插结点作为根结点插入到空树中。

若二叉排序树非空，将待插结点的关键字与根结点的关键字比较：

若相等，则说明树中已有该结点，无须再插入；

若待插结点的关键字 $<$ 根结点的关键字时，则将待插结点插入到根的左子树；

若待插结点的关键字 $>$ 根结点的关键字时，则将待插结点插入到根的右子树；

在左子树、右子树中的插入过程与在树中的插入过程相同，如此下去，直到待插结点作为一个**新的树叶**插入到二叉排序树，或发现树中已有该结点为止。



4、二叉排序树的生成

生成：对于给定的记录关键字序列，首先从空的二叉排序树开始，每输入一个关键字，就建立一个新的结点插入到当前已生成的二叉排序树中，插入规则同前。

二叉排序树的重要特征：

按中序遍历二叉排序树得到一个递增有序的序列

对于给定的记录关键字集合，不同的输入顺序可以构造出不同形态的二叉排序树，共有：

$$\frac{1}{n+1} C_{2n}^n$$



5、二叉排序树的删除

分三种情况讨论：

设待删结点为P，其双亲为F，
且P是F的左孩子

若待删结点P是叶子结点，则直接删去此结点

若待删结点P只有左子树无右子树，或只有右子树无左子树，则用其左子树的根或右子树的根代替被删结点P。

若待删结点P既有左子树又有右子树，则不能简单处理，有以下处理方法：（设P的中序遍历的**前驱**结点为S， $S \rightarrow rchild = \text{NULL}$ ）

用S替代P，S原来的左子树成为S双亲Q的右子树。



6、二叉排序树查找效率分析

查找效率不仅与树中结点个数有关，还与树的形态有关

最坏情况：当关键字有序时，构成的二叉排序树蜕变为单枝树，树深为 n

$$ASL_{ss} = (n+1)/2 \quad (\text{等概下})$$

最好情况：二叉树的形态与折半查找的判定树相同

$$ASL_{ss} = O(\log_2 n) \quad (\text{等概下})$$



二、B-树和B+树

(一)、B-树

定义：一棵m阶的B-树，或为空树；若树不空，则为满足下列特性的m叉树：

$2 \leq \text{根结点子树数} \leq m$

$\lceil m/2 \rceil \leq \text{其他非叶子结点子树数} \leq m$

所有非终端结点结构： $nA_0K_1A_1K_2A_2 \dots K_nA_n$

其中： n 为结点关键字个数， $\lceil m/2 \rceil - 1 \leq n \leq m-1$

$A_0 \sim A_n$ ：指向子树根结点的指针

$K_1 \sim K_n$ ：结点关键字

在同一结点上： $K_i < K_{i+1}$ ，且同一结点上关键字有序

对于“ $A_{i-1}K_iA_i$ ”有： K_i 大于 A_{i-1} 所指结点上的任一关键字

K_i 小于 A_i 所指结点上的任一关键字

所有叶子结点均在同一层上，且不带信息。

2、B-树的运算

(1) B-树的查找运算

在B-树中查找给定的关键字的方法：

首先从根结点开始，在根结点所包含的关键字中查找给定的关键字 K （采用顺序查找或折半查找），直到 $K_{i-1} < K \leq K_i$ 或 $K > K_n$ 。若找到等于给定值 K 的关键字 $K = K_i$ ，则查找成功；否则，一定可以确定要查的关键字是在某个 K_{i-1} 和 K_i 之间（关键字有序），于是取 A_{i-1} 所指向的结点继续查找；如此重复下去，直到找到或指针 A_{i-1} 为空时（找到叶子），查找失败。

查找成功：可在根或非叶子结点。

查找失败：从根走到某一叶子结点。



B-树查找效率分析：

在B-树上进行查找包含两种基本操作。

在B-树上找结点。

在结点中找关键字。

设m阶B-树中具有N个关键字，因此有N+1个树叶，所以 $2^{\lceil m/2 \rceil - 1} \leq N+1 \Rightarrow 1 \leq \log_{\lceil m/2 \rceil} ((N+1)/2) + 1$ 。

$O(\log_{\lceil m/2 \rceil} N)$ 。

B-树的查找效率是相当高的。



(2) B-树的插入运算

B-树插入规则:

m阶B-树, 若叶子结点处于第L+1层时, 插入的关键字总是进入第L层的结点。

若结点的关键字个数 $< m-1$ 时, 则直接插入。

若结点的关键字个数 $= m-1$ 时, 则插入后分裂结点。



(3) B-树的删除运算

B-树删除原则

m阶B-树，叶结点处于第L+1层，若被删关键字位于非L层结点中，对于关键字 K_i 找 A_i 所指子树中最小关键字互换，使 K_i 到L层结点中。所以仅考虑删L层结点：

若结点的关键字个数 $> \lceil m/2 \rceil - 1$ ，则直接删除。

若结点的关键字个数 $= \lceil m/2 \rceil - 1$ ，则首先向兄弟结点借关键字：

若允许，则删除完成。

若借不到，则进行结点合并。合并操作可能一直到根结点，使树的深度减少一层。



§ 4. 哈希表 (Hash)

哈希表采用的是直接存取技术



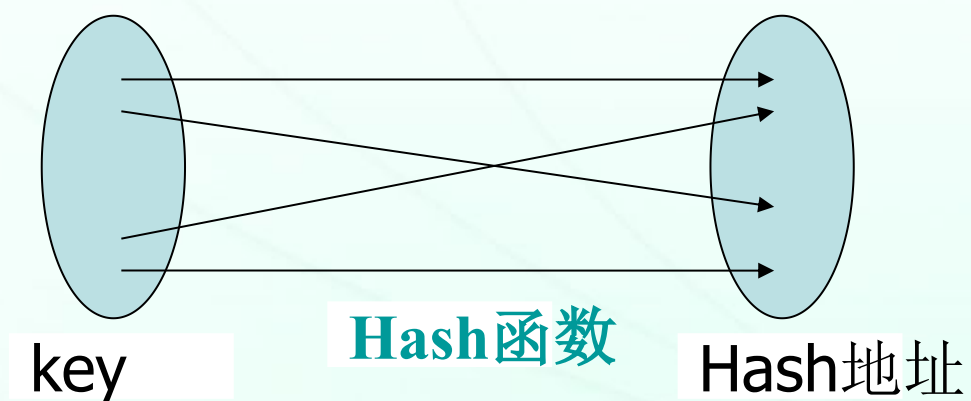
一、哈希表查找的基本概念

1、基本思想

不直接比较，通过哈希函数建立关键字到哈希表的映射关系（可能多对一）。

建立哈希表时。

在哈希表中查找记录时。



二、解决冲突的方法

1、开放定址法：

当冲突发生时，使用某种方法在Hash表中形成一个探查序列，然后沿着此探查序列逐个单元地查找，直到找到给定关键字或碰到一个开放的地址（即地址单元为空）为止。

插入时：碰到开放的地址，则将待查关键字插入。

查找时：碰到开放的地址，则说明查找失败。



(1)、线性探测再散列

$$d_i = (d + i) \bmod m$$

$$i = 1, 2, \dots, m-1$$

m为Hash表表长

d为冲突产生时的Hash地址

通过 i 取值的线性变化构造探查序列

特点：线性



(2) 二次探测再散列

$$\begin{cases} d_{2i-1} = (d + i^2) \bmod m \\ d_{2i} = (d - i^2) \bmod m \end{cases} \quad 1 \leq i \leq (m-1)/2$$

特点:

将同义词来回散列在第一次产生冲突时的Hash地址的两端

探测序列是跳跃似的

减少堆积（二次聚集）的发生

缺点：不易探测到整个Hash表空间



(3) 伪随机探测再散列

$$d_i = (d + R_i) \bmod m$$

m 为Hash表长

d 为冲突产生时的Hash地址

R_i : $R_1, R_2, \dots, R_{m-1}$ 是一个伪随机数序列



2、再哈希法

当冲突产生时，通过再计算另一个哈希函数地址解决冲突。即为构造不同的哈希函数。

$$d_i = RH_i(K)$$

$$i=1, 2, \dots, k$$

其中： $RH_i(K) \neq RH_j(K)$, 即 RH_i 均是不同的Hash函数。

优点：不易产生堆积或聚集。

缺点：计算量大。



3、链地址法

将所有关键字为同义词的记录存储在单一单链表中。

假设Hash函数产生的哈希地址在区间 $0 \sim m-1$ 上，则将Hash表定义为一个由 m 个头指针组成的指针数组 $chainHash[m]$ ，凡Hash地址为 i 的记录都插入到以数组元素 $chainHash[i]$ 为头指针的链表中，并保持同义词在同一线性链表中按关键字有序。

优缺点

优点：不会产生堆积；空间是动态申请的，适用于造表前无法确定表长的情况。

缺点：空间开销大。



三、哈希表的算法分析

哈希表的装填因子

$$\alpha = \frac{\text{表中填入的记录数}}{\text{哈希表的长度}} = \frac{n}{m}$$

α 体现哈希表的装满程度

哈希表的查找效率的决定因素

哈希函数

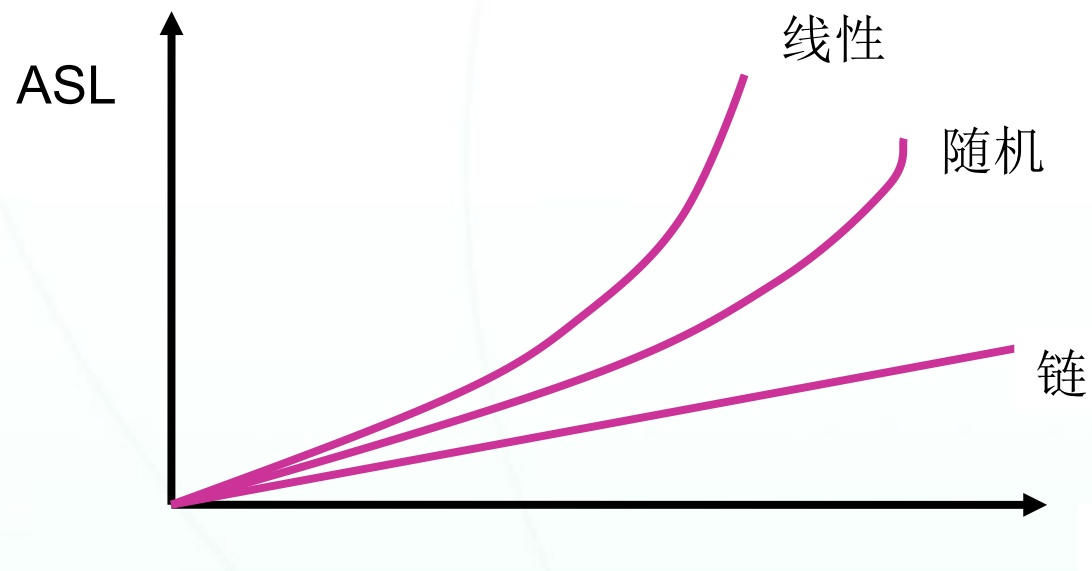
解决冲突的方法

哈希表的装填因子



假设Hash函数均匀的前提下，不同的解决冲突的方法得到的Hash表的ASL不同，且不是表中关键字个数n的函数，而是装填因子 α 的函数。如下表所示：

解决冲突的方法	平均查找长度ASL	
	成功查找	不成功查找
线性探测法	$(1+1/(1-\alpha))/2$	$(1+1/(1-\alpha)^2)/2$
二次探测、随机探测、再哈希法	$-\ln(1-\alpha)/\alpha$	$1/(1-\alpha)$
链地址法	$1 + \alpha / 2$	$\alpha + \exp(-\alpha)$



α 越小，发生冲突的可能性越小。

α 越大，发生冲突的可能性越大。

因此只要 α 选择合适，不管 n 多大，哈希表的平均查找长度就会限定在一个范围之内。

(本章结束)

第十章 文件

基本概念

文件：是性质相同的记录的集合

1. 文件的逻辑结构及操作
2. 文件的存储结构：顺序文件、索引文件、散列文件和多关键字文件



顺序文件

顺序文件的顺序存取

顺序文件的随机存取

顺序文件按关键字存取

优点：顺序存取速度快



索引文件

索引文件有索引表 and 主文件两部分构成。

检索方式：直接存取或按关键字存取



ISAM文件

索引顺序存取方法ISAM (Indexed Sequential Access Method):
一种专为磁盘存取设计的索引顺序文件组织方式。

组成: 多级主索引、柱面索引、磁道索引和主文件。



VSAM 文件 (virtual Storage Access Method)

索引顺序文件——动态索引结构

一. B+树的定义:

一棵 m 阶的B树, 或为空树, 或是满足下列条件的 m 叉树:

- (1) 树中每个结点至多有 m 棵子树;
- (2) 除根之外的所有分支结点至少有 $\lceil m/2 \rceil$ 棵子树;
- (3) 若根结点不是叶子结点, 则至少有两棵子树;
- (4) 有 n 棵子树的结点有 n 个关键码;

(5) 叶结点中存放数据文件中记录的关键码及指向该记录的指针 (或存放数据文件分块后每块的最大关键码及指向该块的指针)。叶结点按关键码值大小顺序链接。可以把每个叶结点看成一个基本索引块。

(6) 所有分支结点可看成是索引的索引，使结点中仅包含它的各子结点中最大（或最小）关键码及指向子结点的指针。

一棵 m 阶的 B^+ 树和 B -树的差异在于：

(A) 有 n 棵子树的结点中含有 n 个关键码；

(B) 所有的叶子结点中包含了全部关键码的信息，

及指向含这些关键码记录的指针，且叶子结点的本身依关键码的大小从小到大顺序链接。

(C) 所有的非终端结点可以看成是索引部分，结点中仅含有其子树（根结点）中最大（或最小）关键码。

通常在 B^+ 树上有两个头指针，一个指向根结点，另一个指向关键码最小的叶子结点。



散列文件

散列文件：用散列技术组织成的文件，组织方法类似散列表，存储介质是外存储器。

更多优质自考资料尽在百度贴吧自考乐园俱乐部
(<http://tieba.baidu.com/club/5346389>)

欢迎♥加入...欢迎♥交流...止不住的惊喜等着你.....



多关键字文件

多关键字文件：

建立一系列的次关键字索引，这样建立的文件即为多关键字文件。

多关键字文件的组织方法：

多重表文件

倒排文件



自考乐园诚与天下自考人共勉

☆自考乐园---心境随缘，诚与天下自考人共勉！！

☆自考乐园---分享快乐，你的快乐老家！！

☆自考乐园---引领成功，你的精神乐园！！

☆自考乐园---心境随缘，诚与天下自考人共勉！！

☆自考乐园---分享快乐，你的快乐老家！！ ☆自考乐园---引领成功，你的精神乐园！！

自考乐园俱乐部，专注于自考，致力于成为全国最全，最优的自考学习交流，资料共享平台.....

各位自考的朋友：

大家好，我先介绍一下自己吧，我叫杨尚杰，由于种种原因，高考不是很理想，以

至于在08年进入了一所民办专科院校，可是我并不认输，在大学除了积极锻炼自己外，在大一下学期，我依然选择了自考计算机网络专业独立本科，目前已顺利通过了8科，在自考征途中，本人也搜集了很多自考的资料（我会把我的资料上传到百度文库或新浪共享里，把下载网址整理后发到本俱乐部），本着“分享快乐”的观点，本人真诚的欢迎广大自考的朋友（尤其是报考和我相同专业的考生）加入本俱乐部，一起交流，进步，提高.....

? -----



自考乐园简介

自考乐园<http://tieba.baidu.com/club/5346389>

自考乐园,自考学习交流、资料共享的好去处!

自考乐园,自考人自己的家园...

更多优质资料尽在百度贴吧俱乐部: 自考乐园, 欢迎加入自考乐园俱乐部, 如有需要自考资料, 请留言, 我们会尽力帮助大家...祝大家自考快乐.....

俱乐部名称: 自考乐园

俱乐部id: 5346389 (请牢记它哦~在百度贴吧的搜索框中输入俱乐部id, 可以直接进入俱乐部)

俱乐部url地址: <http://tieba.baidu.com/club/5346389>

(您也可以通过此url进入俱乐部。)

俱乐部简介:

- ①自考乐园俱乐部, 专注于自考, 致力于成为全国最全, 最优的自考学习交流, 资料共享平台.....我们正寻找特别的你亲情加入.....
- ②本人自考计算机网络独立本科已过8科, 真诚欢迎自考的朋友进来一起交流, 进步, 提高.....欢迎加入自考乐园俱乐部, 如有需要自考资料, 请留言, 我们会尽力帮助大家...祝大家自考快乐.
- ③自考乐园俱乐部, 诚邀天下自考人共勉!!! 欢迎加入自考乐园俱乐部.....

☆☆☆与千万自考生同行，你准备好了吗？

你希望在自考的征途中，有一群和你志同道合的人同行吗？

你愿意在漫长的自考岁月中，有一群人和你分享快乐，分担忧愁吗？

你渴望在一个人奋斗时，有一群人在背后默默支持你吗？

你是否也一直在苦苦寻找这样一个平台，一群志同道合的人，一片积极向上的心，一个共同的追求，一个诚挚的鼓励，一个坚实的支持.....对！！！！就是这里！！！！这里有你想要的一切.....



☆☆☆与千万自考生同行，你准备好了吗？

与志者同行，你也将成为志者！！！

与成功者同行，你也将获得成功！！！

与千万自考生同行，你准备好了吗？？？

与千万自考生同行，你做好了准备吗？？？

今天我们诚挚的发出邀请，真诚的欢迎广大自考的朋友（尤其是和我报考相同专业的朋友）加入本俱乐部，一起交流，进步，提高.....我们正寻找特别的你亲?情加入.....

最后预祝所有的朋友:

自考快乐

天天有份好心情！！！

-----杨尚杰于2010.4.19





谢谢大家

自考乐园
祝同学们自考快乐考试成功!!!

